

Chunk 1: Summary of the code

*# Herein are all 40 code chunks used for the research paper "Escaping Inequality in
→ India." They are organized like so:*

Chunk 1: Summary of the code

Chunk 2: Introduction

Chunk 3: Define functions to read in short 8.3 filenames

read_sav_short(long_path)

read_csv_short(long_path)

read_excel_short(long_path, sheet)

Chunk 4: Import key data files

District boundaries and regions

Chunk 5: Match districts: Extract district info from key datasets

Diagnose and fix issues

Chunk 6: Match districts: Construct district tracking dfs

Construct district_tracker df

Diagnose and correct mistakes

Chunk 7: Participation model: Cleaning and prep

District-level aggregates for imputation

Chunk 8: Participation model: Is missingness associated with observed covariates

All variables: Variables/Regions with NAs

Correlation matrix

Logistic per missing variable

check_missing_logit_parallel(df, miss_vars, covars, method_p)

fit_one(m)

Chunk 9: Participation model: Estimation and IMR

Chunk 10: Participation model: Derive AME

Chunk 11: Participation model: Set up results table

```

# Chunk 12: Participation model: Make summary tables

# Chunk 13: Construct 2007-08 measures
# Diagnose issues

# Chunk 14: Construct 2017-18 measures
# Chunk 15: Construct 2001 measure (IV)

# Chunk 16: Match districts: Test joining methods
# evaluate_distances(pairs, methods, thresholds, col1, col2)
# Helper vectors

# Chunk 17: Match districts: Define joining functions
# fuzzy_join_sequence(df1, df2, dist1, state1, dist2, state2, methods, thresholds,
#   ↪ mode)
# merge_dfs_into_tracker(dfs, tracker, years_of_interest, flag)

# Chunk 18: Match districts: Manually fix errors
# Chunk 19: Match districts: Run the joining functions
# Chunk 20: Match districts: Diagnose errors
# unmatched_rows(dfs, tracker, years_of_interest)
# Compare outcomes with different tracker dfs
# Correct even more NAs

# Chunk 21: Match districts: Join with geometry data
# Compare districts in district_tracker merge vs. district_timeseries merge

# Chunk 22: Geospatial: Make maps
# Save the maps

# Chunk 23: Geospatial: Make map collages
# Chunk 24: Geospatial: Neighbor list construction

# Chunk 25: 2SLS: Controls and 2SLS formula
# Control vectors
# IV Formula
# make_iv_formula(dep, endog, exog, inst)

```

```

# Chunk 26: 2SLS: Baseline 2SLS models
# Chunk 27: 2SLS: Make summary tables
# Chunk 28: Multicollinearity check

# Chunk 29: Geospatial: Spatial autocorrelation tests
# Chunk 30: Spatial-2SLS model attempts

# Chunks 31-40: [Output for paper]

# For easier navigation, use one of the following:
# 1) Search (e.g., using ctrl/cmd+F) for specific chunk, section, and/or function names.
# 2) Open the table of contents or document outline in your PDF viewer's navigation pane.

```

Chunk 2: Introduction

```

knitr::opts_chunk$set(echo = TRUE, # Show code
                      message=FALSE,
                      warning=FALSE,
                      include=TRUE, # Include code and output
                      cache = TRUE,
                      cache.lazy = FALSE # Since we have large objects (see
                      ↪ https://yihui.org/knitr/options/#chunk-options)
                      )

options(tinytex.verbose = TRUE) # Leave uncommented to help troubleshoot when knitting

library(this.path) # here() function
library(haven) # Read in SPSS, Stata, and SAS files
library(readxl) # Read in .xls and .xlsx files
library(readODS) # Read in .ods files

```

```

library(sf) # For geospatial data in dataframes
library(spdep) # Spatial weights matrices

library(scales) # pretty_breaks(), comma()
library(formatdown) # format_numbers()

library(forcats) # Manipulate factor levels
library(survey) # Sample-weighted probit estimation
# library(sampleSelection) # Extract inverse Mills ratio
library(marginaleffects) # "Quickly" derive average marginal effects from probit

library(broom) # tidy(): glm objects --> tidy df of coefs
library(parallel) # mclapply(), parLapply() parallelization
# library(future) # Parallelize
# library(future.apply) # Parallelize more _apply() functions

library(stargazer) # Regression table outputs
# library(xtable) # For function which automates regression table creation

library(DescTools) # Calculate Gini coefficients

library(fuzzyjoin) # Join df's over approximate matches
library(stringdist) # Approximately match strings

library(purrr) # Mapping and manipulating dfs

library(tmap) # Make maps
library(kableExtra) # Make tables
library(modelsummary) # Make more tables

library(AER) # Run 2SLS regressions
library(sandwich) # HC and clustered SEs

```

```

library(lmtest) # Run custom Wald tests

library(magick) # Make collages

library(tidyverse)

# From an earlier version of this document, kept just in case:
# library(reticulate) # To access Python and read in weirdly formatted PDF tables
# py_install(c("pandas", "tabula-py", "openpyxl", "JPype1"), pip = TRUE) # Ditto. If this
  ↳ didn't work, uncomment the following lines
# py_install("pandas", pip = TRUE)
# py_install("tabula-py", pip = TRUE)
# py_install("openpyxl", pip = TRUE)
# py_install("JPype1", pip = TRUE)

setwd(here())

```

Chunk 3: Define functions to read in short 8.3 filenames

```

# ---TROUBLESHOOTING---

# If getting "file does not exist" or similar errors:
# Switch the function used to read in the file e.g., if the relevant file is being read
  ↳ in using read_sav() or read_sav_short(), switch to read_sav_short() or read_sav()
  ↳ respectively.

# If that didn't work and you're not using Windows:
# Either modify this code chunk for your OS (note the "if" statement at the start of each
  ↳ function) or manually shorten the error-inducing file paths.

```

```

# ---

# Purpose of this chunk:
# To ensure R can identify and read in all necessary files.
# This project found me making frequent use of file paths over 260 characters in length.
  ↳ Out of my naivete, I didn't realize Windows File Explorer automatically shortens such
  ↳ file paths by rewriting them in the 8.3 filename convention--meaning R can no longer
  ↳ access their standard "long name." Conversely, functions like read_sav()
  ↳ automatically lengthen 8.3 filenames via functions like normalizePath(), turning the
  ↳ true 8.3 file path into a long name which, from R's and (thus our) perspective, is
  ↳ useless.
# This code chunk turns our long name inputs into 8.3 filenames while tricking the
  ↳ read_*()-type functions into not turning them back.

# Context for nerds:
# In my Registry Editor, under
  ↳ Computer\HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem, I had set
  ↳ the LongPathsEnabled value to 1 (which enables Windows to use long file paths) and
  ↳ the NtfsDisable8dot3NameCreation value to 2. I had also changed Group Policy to
  ↳ "Enable Win32 long paths".
# But none of that mattered! Windows File Explorer lacks a longPathAware entry in its
  ↳ .exe file's manifest--so despite LongPathsEnabled=1, my file paths were still capped
  ↳ at 260 characters. Ergo this code chunk.
# It's possible that setting NtfsDisable8dot3NameCreation=0 would've prevented all of
  ↳ this code from being necessary. But I'd rather not force replicators to change their
  ↳ registry settings.

# ---

# Goal: Make an alternative read_sav() which accepts 8.3 filenames

```

```

read_sav_short <- function(long_path, ...) {

  # shell() only exists in Windows; so if not running on Windows, just use read_sav()
  ↪ normally
  if(Sys.info()['sysname'] != "Windows"){
    return(read_sav(long_path, ...))
  }

  # Get the 8.3 alias of the file's absolute path from shell
  short_path <- shell(
    sprintf('for %I in ("%s") do @echo %~sI', long_path),
    intern = TRUE
  ) # Note: Use double percent signs (%%) to prevent CMD shell from thinking each
  ↪ percent sign is a delimiter for a variable name.

  # Open a connection ("con") in binary mode ("rb" = "read binary"; SPSS files are binary
  ↪ i.e. non-text files, and thus best read in as such) to the 8.3 filename
  con <- file(short_path, "rb")

  # Save space by closing the connection when this function finishes executing
  on.exit(close(con))

  # Pass this open connection to read_sav() so it doesn't convert the 8.3 filename back
  ↪ into the long filename
  data <- read_sav(con, ...)
  return(data)

  # There's something weird happening here. In my File Explorer, the entire absolute path
  ↪ is not an 8.3 alias: only the relative path (the path from the working directory,
  ↪ or here, the names of the file and the folder containing it) is. Yet this code only
  ↪ works because it feeds a connection, opened via file() using an absolute path's 8.3
  ↪ alias, directly into read_sav(). This should *automatically happen in read_sav()
  ↪ anyways*!!! read_sav(path) calls datasource(path) calls
  ↪ readr:::standardise_path(path) (an internal function, see ?`:::` and
  ↪ https://github.com/tidyverse/readr/blob/96ddac314b47402bc63e1f81c149c463cf58e3da/R
  ↪ /source.R#L119) calls readr:::detect_compression(path) calls readBin(path) calls

```

```

}

# ---

# Goal: Likewise, now for reading in CSV files

read_csv_short <- function(long_path, ...){

  # shell() only exists in Windows
  if(Sys.info()['sysname'] != "Windows"){
    return(read_csv(long_path, ...))
  }

  short_path <- shell(
    sprintf('for %I in ("%s") do @echo %%~sI', long_path),
    intern = TRUE
  )

  con <- file(short_path, "rb") # CSVs are text, not binary, files. But read_csv() feeds
  ↪ its inputs into vroom() (https://github.com/tidyverse/readr/blob/96ddac314b47402bc63e1f81c149c463cf58e3da/R/read\_delim.R#L253), which apparently can only read from a
  ↪ binary connection? Otherwise vroom() calls vroom_() (https://github.com/tidyverse/vroom/blob/73c90c4fe490c0588b20ac527c40fcb1c683683e/src/vroom.cc) which in turn calls a
  ↪ function which looks very similar to the source code of readBin()
  ↪ (https://github.com/SurajGupta/r-source/blob/a28e609e72ed7c47f6ddfb86c85279a0750f0b7/src/library/base/R/connections.R#L219). The vroom package seems to only call
  ↪ readBin() at vroom:::R_ReadConnection() (https://github.com/tidyverse/vroom/blob/73c90c4fe490c0588b20ac527c40fcb1c683683e/src/connection.h#L52), though I'm not sure how
  ↪ the call chain could go from vroom_() to R_ReadConnection()--nor am I sure as to why
  ↪ read_csv() reads in text files as binary. Perhaps read_csv() reads in the raw bytes
  ↪ using readBin() before applying its own parser to the text?

  on.exit(close(con))

```



```

data <- read_csv(con, ...)
return(data)
}

# ---

# For Excel files

# Note: read_excel() only accepts file paths as strings, not connections (see
→ https://github.com/tidyverse/readxl/blob/main/R/read\_excel.R#L183 and
→ https://github.com/tidyverse/readxl/blob/2a7f6efd8562ca5beba0b34933c8e4a9332ed6a8/src/Read.cpp#L86). It seems like all similar functions are the same, too.

read_excel_short <- function(long_path, sheet = 1, ...){ # We want sheet = 1 to be the
→ default for all commands within this function

  # shell() only exists in Windows
  if(Sys.info()['sysname'] != "Windows"){
    return(read_excel(long_path, sheet = sheet, ...))
  }

  # Split long_path into its two components: the folder and the file
  long_folder <- dirname(long_path)
  long_file <- basename(long_path)

  # Extract the 8.3 alias of the folder's absolute path
  short_folder <- shell(
    sprintf('for %I in ("%s") do @echo %~sI', long_folder),
    intern = TRUE
  )

```

There seems to be no similar way to get the file's 8.3 alias safely. To bypass this,
→ *we run a directory call on the folder's 8.3 name, which lists out all the files in*
→ *the folder, identified by both their long and short names.*

```
dir_output <- shell(  
  sprintf('dir /x "%s"', short_folder),  
  intern = TRUE  
)
```

Identify the relevant file's row as the one which includes an exact match (ergo
→ *"fixed", meaning no regex parsing) to long_file and then store it (ergo "value").*

```
candidate_line <- grep(long_file, dir_output, value = TRUE, fixed = TRUE) # grep()  
→ searches through a vector of strings and returns the indices or values of elements  
→ which contain a match.
```

To identify the file's 8.3 name from this row, see
→ *<https://tomgalvin.uk/blog/gen/2015/06/09/filenames/>: a file name of 8 characters or*
→ *more is upper-cased, stripped of its invalid characters (e.g., spaces), truncated*
→ *to the first 6 characters of its basename, and then followed by a tilde, a single*
→ *digit, and the first 3 characters of its upper-cased file extension en route to*
→ *becoming an 8.3 filename.*

```
pattern <- "([A-Z0-9-]{1,8}~[0-9]\\.[A-Z0-9]{1,3})" # "(...)" = capturing group, though  
→ only used here for cleanliness; "[A-Z0-9]{1,8}" = match with 1-8 uppercase letters or  
→ digits; "~" = match with a tilde; "[0-9]" = match with exactly one digit; "\\." =  
→ match with a period (as "\" is an escape character in both R and regex, meaning "\\."   
→ in R = "." in regex = "." in text)
```

Note that, excluding the working directory, 8.3 filenames must be weakly less than 8
→ *characters. Ergo "{1,8}"*

```
match <- regexpr(pattern, candidate_line, perl = TRUE) #regexpr() searches a single  
→ string and returns an integer vector indicating the starting position of the first  
→ match and the length of the matched text10 Perl-style matching is used for its speed  
→ (per the regexpr() documentation) and flexibility (see  
→ https://gist.github.com/CMCDragonkai/6c933f4a7d713ef712145c5eb94a1816) over the
```

```

# I'm not entirely sure why this code is working so well; just in case, I'm including
↪ error codes for your benefit.
if (match[1] == -1){
  stop("Could not find the file's 8.3 alias from the folder's directory listing.")
}

# Extract the file's 8.3 alias from this match data
short_file <- regmatches(candidate_line, match)

# Construct the absolute short path using short names
abs_short <- file.path(short_folder, short_file)

# Copy the file to a temporary one so that our file readers won't re-normalize its path
ext <- tolower(tools::file_ext(short_file)) # Extract the file's extension
temp_file <- tempfile(fileext = paste0(".", ext))

# In case your computer is messing up in ways mine wasn't.
if (!file.copy(abs_short, temp_file, overwrite = TRUE)){
  stop("File failed to copy from its short path to a temporary location.")
}

# Using read_excel() would return an error from libxls, the C library. So we turn to
↪ other packages instead.

# Only use xlsx package to read in .xls to limit the need for Java
# Load the packages via namespace so they only need to be installed if needed. Use
↪ xlsx::read.xlsx for .xls files.

# Read in from the temporary file
if (ext %in% c("xls", "xlsb")) {

  # If needed namespace is installed, read in the file

```

```

if (!requireNamespace("xlsx", quietly = TRUE))
  stop("Install the \"xlsx\" package and Java (see
    ↪ https://www.java.com/en/download/manual.jsp).")
data <- xlsx::read.xlsx(temp_file, sheetIndex = sheet, ...)
}

# Repeat
else if (ext == "xlsx") {
  if (!requireNamespace("openxlsx", quietly = TRUE)){
    stop("Install the \"openxlsx\" package.")
  }
  data <- openxlsx::read.xlsx(temp_file, sheet = sheet, ...)
}

return(data)
}

```

Chunk 4: Import key data files

```

#### From NSS 2007-08, 64th Round

data_folder <- "NSS 2007-08 Participation and Expenditure in Education 64th Round"

#edu0708b12 <- read_sav(file.path(data_folder, "Block-1 & 2 Identification of sample
  ↪ household.sav"))
edu0708b3 <- read_sav(file.path(data_folder, "Block-3 Household characteristics.sav"))
edu0708b4 <- read_sav_short(file.path(data_folder, "Block-4 Demographic and other
  ↪ particulars of household members.sav"))
edu0708b5 <- read_sav_short(file.path(data_folder, "Block-5 Education particulars of
  ↪ those aged 5-29 years who are currently attending primary level and above.sav"))

```

```

edu0708b6 <- read_sav(file.path(data_folder, "Block-6 Particulars of private
  ↪ expend.sav"))

#edu0708b7 <- read_sav_short(file.path(data_folder, "Block-7 Particulars of currently
  ↪ not attending persons aged 5-29 years.sav"))

data_folder <- "NSS 2007-08 Household Consumer Expenditure Survey 64th Round"

cons0708hhchar <- read_sav(file.path(data_folder, "Household Characteristics.sav"))

# ---

#### From NSS 2017-18 Household Social Consumption Education, 75th Round

data_folder <- "NSS 2017-18 Household Social Consumption Education 75th Round Data July
  ↪ 2017 - June 2018"

#edu1718b1211 <- read_sav_short(file.path(data_folder, "Blocks 1, 2 and 11 -
  ↪ Identification of sample household.sav"))
edu1718b3 <- read_sav_short(file.path(data_folder, "Block 3 - Household
  ↪ characteristics.sav"))
#edu1718b31 <- read_sav_short(file.path(data_folder, "Block 3.1 - Details of erstwhile
  ↪ household members of age 3 to 35 years currently attending education.sav"))
#edu1718b4 <- read_sav_short(file.path(data_folder, "Block 4 - Demographic and other
  ↪ particulars of household members.sav"))

# Recall: Long filenames were used so that updated versions of data files could be
  ↪ downloaded and easily plugged in.

# ---

```

```

#### From the Census of India 2001
data_folder <- "Indian Census 2001"

# start.time <- Sys.time()
# 35 files will be read in. Preallocate list to store their cleaned data
files <- list.files(
  data_folder,
  pattern = "^PC01_C16_\\d{2}\\\\.xls$",
  full.names = TRUE
)
result_list <- vector("list", length(files))

for (i in seq_along(files)) {
  file_path <- files[i]
  temp <- read_excel(file_path, skip = 6, col_names = FALSE)

  # Explicitly set column names
  colnames(temp) <- c("table", "state_code", "district_code", "tehsil_code", "area_name",
    "mother_tongue_code", "mother_tongue", "spkr_tot", "m_spkr_tot",
    "f_spkr_tot", "spkr_urban", "m_spkr_urban", "f_spkr_urban",
    "spkr_rural", "m_spkr_rural", "f_spkr_rural")

  # Extract the state name:
  # Find the first row in the area_name column that begins with "State - "
  state_val <- temp$area_name[grepl("^State - ", temp$area_name)]
  if(length(state_val) == 0) {
    state <- NA_character_
  } else {
    state <- state_val[1] %>%
      # Remove the "State - " prefix

```

```

    str_remove("^State -\\s*") %>%
    # Remove four-digit suffix code
    str_remove("\\s*\\d{4}$") %>%
    # Remove leading/trailing spaces. Squish internal spaces
    str_squish() %>%
    # Convert to title case
    str_to_title()
  }

# Filter the dataframe for rows where both:
# 1. "mother_tongue_code" ends with "0"
# 2. "area_name" begins with "District - "
temp <- temp %>%
  filter(str_detect(as.character(mother_tongue_code), "0$"),
         str_detect(area_name, "^District - "))

# Create "district" and "state" columns
temp <- temp %>%
  mutate(
    # Create "district" by cleaning "area_name"
    district = area_name %>%
      str_remove("^District -\\s*") %>%
      str_remove("\\s*\\d{4}$") %>%
      str_squish() %>%
      # Remove all trailing non-letters
      str_replace("[^[:alpha:]]+$", ""),

    # Create "state" via extracted state name
    state = state
  )

# Clean "mother_tongue" column

```

```

temp <- temp %>%
  mutate(mother_tongue = mother_tongue %>%
    str_remove("^\\d{1,3}\\s+") %>% # Remove leading 1-3 digits and a space
    str_to_title()) # Title case

# Save cleaned df in list
result_list[[i]] <- temp
}

# Combine cleaned dfs
census01 <- bind_rows(result_list)

# end.time <- Sys.time()
# end.time - start.time
# For us, above order (clean files, then bind) was 1.48 secs faster than alternatives.

# Uncomment below to visually verify results
# census01 %>% select(-c(table, tehsil_code, area_name, mother_tongue_code)) %>%
  ↪ group_by(state) %>% distinct(district) %>% View()

# ---

#### District boundaries and regions ####
# From @meyersIndiaOfficialBoundaries2020 via @bhatiaMergingUpdatedDistrictlevel2020
data_folder <- "District Boundaries 2020"

district_bnds_20 <- st_read(
  file.path(data_folder, "district", "in_district.shp"),
  quiet = TRUE) %>%
  rename(district_20 = dtname,
    state_20 = stname) %>%

```



```

mutate(state_20 = str_to_title(state_20))

# Lookup table: state --> region
region_df <- tribble(
  ~state_20, ~region,
  "Andaman and Nicobar Islands", "South",
  "Andhra Pradesh", "South",
  "Arunachal Pradesh", "Northeast",
  "Assam", "Northeast",
  "Bihar", "East",
  "Chandigarh", "North",
  "Chhattisgarh", "Central",
  "Dadra and Nagar Haveli and Daman and Diu", "West",
  "Delhi", "North",
  "Goa", "West",
  "Gujarat", "West",
  "Haryana", "North",
  "Himachal Pradesh", "North",
  "Jammu and Kashmir", "North",
  "Ladakh", "North",
  "Jharkhand", "East",
  "Karnataka", "South",
  "Kerala", "South",
  "Lakshadweep", "South",
  "Madhya Pradesh", "Central",
  "Maharashtra", "West",

```

```

"Manipur", "Northeast",
"Meghalaya", "Northeast",
"Mizoram", "Northeast",
"Nagaland", "Northeast",
"Odisha", "East",
"Puducherry", "South",
"Punjab", "North",
"Rajasthan", "West",
"Sikkim", "Northeast",
"Tamil Nadu", "South",
"Telangana", "South",
"Tripura", "Northeast",
"Uttar Pradesh", "North",
"Uttarakhand", "North",
"West Bengal", "East"
)

region_df <- region_df %>%
  mutate(region = as.factor(region))

#edu1718b5 <- read_sav_short(file.path(data_folder, "Block 5 - Education particulars on
↪ basic course of the persons of age 3 to 35 years who are currently attending
↪ education.sav"))

#edu1718b6 <- read_sav_short(file.path(data_folder, "Block 6 - Particulars of
↪ expenditure (Rs.) for persons of age 3 to 35 years who are currently attending at
↪ pre-primary and above level.sav"))

```

```
#edu1718b7 <- read_sav_short(file.path(data_folder, "Block 7 - Particulars of currently
  ↪ not attending persons of age 3 to 35 years.sav"))
#edu1718b8 <- read_sav_short(file.path(data_folder, "Block 8 - Particulars of formal
  ↪ vocational or technical training received by household members of age 12 to 59
  ↪ years.sav"))
```

Chunk 5: Match districts: Extract district info from key datasets

```
# From NSS 2007-08 Participation and Expenditure in Education, 64th Round

data_folder <- "NSS 2007-08 Participation and Expenditure in Education 64th Round"

temp <- read_xlsx(file.path(data_folder, "DDI Metadata from Nesstar XML.xlsx")) %>%
  suppressWarnings()

districts_only_0708 <- temp %>%
  filter(name == "district_code") %>%
  select(`ns1:catValu`, `ns1:labl25`) %>%
  distinct() %>%
  filter(str_detect(`ns1:catValu`, "^\\d{5}$")) %>% # ^ = start of string, \\d{5} =
  ↪ exactly five digits, $ = end of string. Filters out codes which are actually for
  ↪ states (2 digits long) or regions (3 digits), keeping only true district codes (5
  ↪ digits).
  rename(district_code_0708 = `ns1:catValu`, district_0708 = `ns1:labl25`) %>%
  mutate(
    state_code_0708 = substr(district_code_0708, 1, 2),
    region_code_0708 = substr(district_code_0708, 1, 3)
  )

regions_only_0708 <- temp %>%
  filter(name == "region_code") %>%
```

```

select(`ns1:catValu`, `ns1:labl25`) %>%
distinct() %>%
filter(str_detect(`ns1:catValu`, "^\\d{3}$")) %>% # ^ = start of string, \\d{3} =
  ⇨ exactly three digits, $ = end of string. Filters for regions (3 digits).
rename(region_code_0708 = `ns1:catValu`, region_0708 = `ns1:labl25`)

states_0708 <- temp %>%
  filter(name == "STATE") %>%
  select(`ns1:catValu`, `ns1:labl25`) %>%
  distinct() %>%
  filter(!is.na(`ns1:labl25`)) %>%
  rename(state_code_0708 = `ns1:catValu`, state_0708 = `ns1:labl25`)

districts_0708 <- left_join(states_0708, districts_only_0708, by = "state_code_0708")

districts_0708 <- left_join(districts_0708, regions_only_0708, by = "region_code_0708")

# From NSS 2017-18 Household Social Consumption Education, 75th Round
data_folder <- "NSS 2017-18 Household Social Consumption Education 75th Round Data July
  ⇨ 2017 - June 2018"

# Huge thanks to Tabula for helping me turn the original PDF table into a CSV!
temp <- read_csv_short(
  file.path(data_folder, "List of Districts NSS 2017-18.csv"),
  col_names = c("sl. no (1)", "state_1718", "region_code_1718", "region_1718", "sl. no
  ⇨ (5)", "district_1718", "district_only_code_1718"),
  col_types = "dcccccc")

```

```

districts_1718 <- temp %>%
  filter(!is.na(district_1718)) %>%
  fill(region_code_1718, .direction = "down") %>% # In the original table, region_code
  ↪ was only given for the first district listed within each region--all other
  ↪ districts had an NA.
  mutate(
    district_only_code_1718 = gsub("[()]", "", district_only_code_1718), # Characters
    ↪ enclosed by square brackets form a character class, a pattern which matches with
    ↪ any character also found inside of the class. So here, we globally substitute
    ↪ ("gsub") an empty string for any character in the district_only_code column which
    ↪ matches the class "[()]" i.e., for any "(" or ")".
    district_code_1718 = paste0(region_code_1718, district_only_code_1718) # Row by row,
    ↪ append the newly de-parenthesized district_only_code to the end of region_code.
  ) %>%
  select(district_code_1718, district_1718) %>%
  mutate(state_code_1718 = substr(district_code_1718, 1, 2)) %>%
  arrange(district_code_1718)

temp <- read_csv(
  file.path(data_folder, "State Codes.csv"),
  col_names = c("state_1718", "state_code_1718"),
  col_types = c("cc"),
  skip = 1)

districts_1718 <- right_join(districts_1718, temp, by = "state_code_1718")

# Drop geometry. Keep JID + keys
districts_20 <- district_bnds_20 %>%
  st_drop_geometry() %>%
  select(JID, district_20, state_20)

```

```

#### Diagnose and fix issues ####

# Fix 2007-08 districts which are assigned multiple codes
# Test for their presence:
# districts_0708 %>% group_by(state_0708) %>% distinct(district_0708) %>% nrow()
# Returns 595
# districts_0708 %>% group_by(state_0708) %>% distinct(district_code_0708) %>% nrow()
# Returns 598
# View the suspected culprits:
repeated_districts_0708 <- districts_0708 %>% group_by(district_0708, state_0708) %>%
  ↪ summarize(n = n_distinct(district_code_0708)) %>% filter(n > 1)
repeats_of_districts_0708 <- districts_0708 %>% filter(
  district_0708 %in% repeated_districts_0708$district_0708,
  state_0708 %in% repeated_districts_0708$state_0708
)
# Clear the innocent suspects:
used_repeats_of_districts_0708 <- edu0708b4 %>% filter(
  district_code %in% repeats_of_districts_0708$district_code_0708
) %>% group_by(district_code) %>% summarize(n())
# Charge the true culprits
unused_repeats_of_districts_0708 <- repeats_of_districts_0708 %>%
  filter(
    !district_code_0708 %in% used_repeats_of_districts_0708$district_code
  )
# Convict them
districts_0708 <- districts_0708 %>% filter(
  !district_code_0708 %in% unused_repeats_of_districts_0708$district_code_0708
)

# Repeat for 2017-18
# # Test for presence of guilt:
# districts_1718 %>% group_by(state_1718) %>% distinct(district_1718) %>% nrow()

```

```

# # Returns 672
# districts_1718 %>% group_by(state_1718) %>% distinct(district_code_1718) %>% nrow()
# # Returns 672
# # All good!

```

Chunk 6: Match districts: Construct district tracking dfs

```

# The districts listed in 2007-08 vs. 2017-18 vary for two reasons: legitimate political
↪ changes (districts may have been created, destroyed, or renamed) and typos. I address
↪ both here.

# Import data on district changes
data_folder = "District Changes Data"

district_timeseries <- read_excel_short(file.path(data_folder, "Time series- State and
↪ Districts Changes -Alluvial 1951-2024.xlsx"))

# Though its extensive time range is informative, there seem to be some typos and factual
↪ errors here (e.g., the 2019 merger of Dadra and Nagar Haveli with Daman and Diu is
↪ not reflected here, and "Dadra" is twice misspelled as "Dadara")

# But not as many mistakes as my original district_tracker, turns out! :D

years_of_interest <- c(2001, 2011, 2024)

district_timeseries <- district_timeseries %>%
  # Select only YYYY-State and YYYY-District for those years
  select(matches(paste0("^(", paste(years_of_interest, collapse="|"),
  ↪ ") [--] (State|District)$")) %>%
  # Rename to state_yy / district_yy
  rename_with(~ tolower(
    str_replace(.x, "(?:\\d{2})(\\d{2}) [--] (State|District)$", "\\2_\\1")

```

))

```
#district_creations <- read_excel_short(file.path(data_folder, "New Districts Created  
→ between 1951-2024.xlsx")) %>%  
→ select(!c.....) # Some files  
→ gained an empty column upon importing
```

```
#district_renamings <- read_excel_short(file.path(data_folder, "Name  
→ Changes_Districts_Indian States_1951-2021.xlsx")) %>%  
→ select(!c.....)
```

```
#district_splits <- read_excel_short(file.path(data_folder, "District Splits and Carve  
→ outs-decadewise 1951-2024.xls")) %>%  
→ select(!c.....)
```

Justify the district_tracker df method of matching

We want to make it so that for each 2017-18 district, either it can be directly matched
→ to a single 2007-08 district, or a single 2007-08 district can be matched to it. To
→ do so, all district changes must have been either clean partitions or name changes of
→ old districts--new districts being carved out of multiple old districts and/or new
→ borders being drawn between old districts make it far harder to equate or claim
→ equality between units of analysis across time.

Up to 2001, @kumarCreatingLongPanels2016 are able to measure the *proportion* of each
→ district's population that was allocated into a new district as a result of district
→ changes, specifically those which are not clean partitions: so carve-outs (of new
→ districts from old districts), mergers, ~~renamings~~renamings, and border shifts. Carve-outs from
→ multiple districts at once and border shifts are the most troublesome for us, as they
→ prevent us from directly matching new districts to a single old district. There does


```

district_carveouts_shifts_9101 <- read_csv_short(
  file.path(data_folder, "District Carve-Outs and Renamings 1961-2001.csv"),
  col_names = c("district_1991", "pop_1991", "district_2001", "pct_01in91",
    ↪ "pct_91in01"),
  col_types = "cccd"
) %>%

fill(district_1991, pop_1991, .direction = c("down")) %>%
filter(!is.na(pct_91in01))

# So we justify directly matching districts by showing that most district changes from
↪ 1991 to 2001 were equivalent to renamings, partitions, or mergers.

plt_district_carveouts_shifts <- district_carveouts_shifts_9101 %>%
  ggplot(aes(x = pct_91in01)) +
  geom_histogram(binwidth = diff(range(district_carveouts_shifts_9101$pct_91in01, na.rm =
    ↪ TRUE))/40, fill = "goldenrod") +
  guides(fill = "none") +
  labs(
    y = "Number of 2001 Districts",
    x = "Percentage of a 1991 District's Population in the 2001 District"
    # title = "Population Reallocations From District Shifts and Carve-Outs, 1991-2001",
    # subtitle = "Number of 2001 Districts Which Absorbed a Percentage of a 1991
    ↪ District's Population"
  )

# From 1991 to 2001, it's evident that district changes which did not involve clean
↪ partitions were rarely associated with changes in district populations. In other
↪ words, most of these changes were approximately equal to name changes. ~[While most
↪ of these changes were actually *precisely* name changes, some were not. See
↪ @kumarCreatingLongPanels2016, for example, to see that certain changes only involved
↪ transfers of uninhabited land.]

```

```

# To get the exact proportion of "other" district changes which are not equivalent to
  ↳ renamings:
# Check pctortion of district changes which allocated less than 100/c or 100 - 100/c of
  ↳ the old district's population to the new district
c = 40

```

```
int_bnds <- district_carveouts_shifts_9101 %>%
```

```

  summarise(
    min = min(pct_91in01),
    max = max(pct_91in01),
    mesh = (max - min) / c,
    int_lower = min + mesh,
    int_upper = max - mesh,
    n = n(),
    n_ext = sum(
      pct_91in01 <= min + mesh |
      pct_91in01 >= max - mesh),
    pct_ext = n_ext/n
  )

```

```

# (Set c = 40) 86% of allocations from a parent district to a child district upon an
  ↳ "other" type of district change involved a transfer of more than 97.5% or less than
  ↳ 2.5% of the population. Meaning 86% of the time, the new district almost precisely
  ↳ matched the old district--the change was effectively a name change.

```

```
carveshift_count <- district_carveouts_shifts_9101 %>%
```

```

  group_by(district_2001) %>%
  arrange(district_2001) %>%
  filter(n() > 1) %>%
  summarize(
    max_pct = max(pct_91in01),

```

```

    min_pct = min(pct_91in01),
    num_parents = n()
  ) %>%
  mutate(
    carveshift = if_else(max_pct >= int_bnds$int_upper & min_pct <= int_bnds$int_lower,
      ↪ 0, 1)
  ) %>%
  ungroup()

# mean(carveshift_count$carveshift)
pct_ok_annoying_districts <- 1 - sum(carveshift_count$carveshift)/int_bnds$n
# = 0.9335106 if c=40, so 93.4% of district changes that were not clean partitions were
↪ equivalent to a name change (allocating more than 97.5% or less than 2.5% of the old
↪ district's population).

# We use this as justification to assume away other types of district changes (namely
↪ district carve-outs and border shifts), allowing us to match each new district to a
↪ single old district or vice versa.

# This is equivalent to what @jaacksIndiaDistrictChanges2020 did in their tracking of
↪ district changes from 2001 to 2020, with the children of district carve-outs only
↪ matched with the one parent who contributed the most to their land area; likewise for
↪ districts following border shifts. We thus use their data below.

# ---

#### Construct district_tracker df ####

# Import data, do preliminary data cleaning

```

```

data_folder = "District Changes Data"

temp <- read_ods(file.path(data_folder, "IndiaDistrictTracker2001to2020.ods"), col_names
  ↪ = FALSE)

temp_names <- tibble(year = as.character(temp[1,])) %>%
  fill(year) %>%
  pull(year)

colnames(temp) <- paste(temp_names, as.character(temp[2,]), sep = "_")

# Set column names as interleaved named vector

years_of_interest <- c("2001", "2005", "2006", "2007", "2008", "2011", "2017", "2018",
  ↪ "2019", "2020") # 2021 excluded due to no data
year_suffixes <- substr(years_of_interest, 3, 4) # "01", "05", etc.

# Order columns by year
state_cols <- paste0("state_", year_suffixes) # paste0() more efficient than paste()
district_cols <- paste0("district_", year_suffixes)

# Interleave columns
interleaved_renamings <- state_cols %>%
  rbind(district_cols) %>% # rbind()
  as.vector()
interleaved_old_names <- paste0(years_of_interest, "_STATENAME") %>%
  rbind(paste0(years_of_interest, "_DISTNAME")) %>%
  as.vector()

# Rename columns
column_renamings <- setNames(

```

```

interleaved_old_names,
interleaved_renamings
)
district_tracker <- temp[-c(1,2), ] %>%
  rename(!!!column_renamings) %>% # `!!!` is the splice operator, to splice and inject
  ↪ arguments from a list into a function call. Note: Base R often needs inject() to
  ↪ accept `!!!`
  select(all_of(names(column_renamings)))

#### Diagnose and correct mistakes ####

#### ...in states/UTs

# View state/UT changes recorded in original data
district_tracker %>%
  rowwise() %>%
  filter(length(unique(c_across(all_of(state_cols)))) > 1) %>%
  ungroup() %>%
  select(all_of(state_cols)) %>%
  invisible()

# Two changes, both from 2019, both first reflected in the 2019 data: the union territory
  ↪ (UT) of Ladakh split from Jammu and Kashmir, and the UT Dadra and Nagar Haveli and
  ↪ Daman and Diu formed from the merger of Dadra and Nagar Haveli with Daman and Diu
# This means four changes were not recorded in the database: renaming Pondicherry (both
  ↪ the district and its UT) to Puducherry in 2006, renaming Uttaranchal to Uttarakhand
  ↪ in 2007, renaming Orissa to Odisha in 2011, and cleaving Telangana out of Andhra
  ↪ Pradesh in 2014.

# Add remaining state/UT changes
district_tracker <- district_tracker %>%
  # For state_xx columns with suffix < "06"

```

```

mutate(across(c(state_01, district_01, state_05, district_05),
               ~ ifelse(. == "Puducherry", "Pondicherry", .))) %>%
# For state_xx columns with suffix < "07"
mutate(across(c(state_01, state_05, state_06),
               ~ ifelse(. == "Uttarakhand", "Uttaranchal", .))) %>%
# For state_xx columns with suffix < "11"
mutate(across(c(state_01, state_05, state_06, state_07, state_08),
               ~ ifelse(. == "Odisha", "Orissa", .))) %>%
# For state_xx columns with suffix < "14"
mutate(across(c(state_01, state_05, state_06, state_07, state_08, state_11),
               ~ ifelse(. == "Telangana", "Andhra Pradesh", .)))
# NOTE: The 07-08 NSS data uses Pondicherry instead of the 2006 name change of
→ Puducherry!! They also (more understandably) use Uttaranchal instead of the 2007 name
→ change Uttarakhand.
#Fix: To join data from 07-08 NSS, must search for matches beyond the sample years.

#### ...in districts

# View number of districts which changed names during a dataset's sampling period
inperiod_district_changes <- district_tracker %>%
  filter(
    district_05 != district_06 |
    district_07 != district_08 |
    district_17 != district_18 |
    district_19 != district_20
  )
# 16 districts :(
# Fix: To join data from multi-year surveys (e.g., 07-08), we'll need to check for
→ matches across all sampled years at least

```

```

# View number of districts each year with the same name despite being in different states

# 1. Filter rows
# For each suffix, we flag rows where the district (for that suffix) appears with more
→ than one distinct state. Then we keep rows where at least one of the suffix
→ comparisons is TRUE.

# Create a list of logical vectors (one per suffix)
keep_list <- lapply(year_suffixes, function(sfx) {

  # Build the column names for this suffix
  state_col <- paste0("state_", sfx)
  district_col <- paste0("district_", sfx)

  # Group the data by district for this suffix and count distinct states
  temp <- district_tracker %>%
    filter(
      !is.na(.data[[district_col]]),
      !is.na(.data[[state_col]])
    ) %>%
    group_by(district = .data[[district_col]]) %>%
    summarise(
      n_states = n_distinct(.data[[state_col]], na.rm = TRUE),
      .groups = "drop"
    )

  # Districts that occur with more than one unique state
  duplicate_districts <- temp %>%
    filter(n_states > 1) %>%
    pull(district)

  # Return a logical vector: TRUE if the district value in the row is in
→ duplicate_districts

```

```

    district_tracker[[district_col]] %in% duplicate_districts
  })

# Combine the logical vectors by taking the rowwise OR (i.e., TRUE if condition holds for
↪ any suffix)
keep_indicator <- Reduce(`|`, keep_list)

# Filter the original data
same_name_districts <- district_tracker[keep_indicator, ]


# 2. Summarize by district for each suffix
# For each suffix, group the filtered data by the district column and count:
# a. the number of rows (n_rows) sharing that district value, and
# b. the number of distinct state values (n_states) in that group.
# Then keep only groups that have more than one unique state.
temp <- lapply(year_suffixes, function(sfx) {
  state_col <- paste0("state_", sfx)
  district_col <- paste0("district_", sfx)
  same_name_districts %>%
    group_by(district_name = .data[[district_col]]) %>%
    summarise(n_rows = n(),
              n_states = n_distinct(.data[[state_col]]),
              .groups = "drop") %>%
    filter(n_states > 1) %>%
    mutate(year_suffix = sfx)
})

# Combine summaries from all suffixes into one data frame
temp <- bind_rows(temp)

# Make it easy to get summary stats
n_same_name_districts <- temp %>%
  group_by(year_suffix) %>%
  summarize(n = n())

# Between min(n_same_name_districts$n) = 3 and max(n_same_name_districts$n) = 5 districts
↪ with the same name in different states in each year of interest

```


Chunk 7: Participation model: Cleaning and prep

```
# Goal: Regress on variables which affect the probability one enrolls in or stays
→ enrolled in school, but which do not otherwise affect district-level outcomes of
→ interest (consumption, wages, etc.)

# Collect data on those aged 5-19 who are in school
selection_df <- edu0708b5 %>%
  filter(AGE <= 19) %>%
  mutate(
    PID = PID,
    district_code = district_code,
    enrolled = 1,
    weight = weight,
    FSU_SL_NO = FSU_SL_NO,
    HHID = HHID,
    STATE = STATE,
    STRATUM = STRATUM,
    SUB_STRATUM_NO = SUB_STRATUM_NO,
    IS_EDU_FREE = factor(
      IS_EDU_FREE,
      levels = c(1,2),
      labels = c("Yes","No")),
    TUTION_FEE_WAIVED = factor(
      if_else(TUTION_FEE_WAIVED %in% c(1,2), "Yes","No"),
      levels = c("Yes","No")), # Note: 1 and 2 = waiver received. 3 = No waiver received.
      → 5 NAs, labeled as No
    RECD_SCHOLARSHIP_STIPEND = factor(
      RECD_SCHOLARSHIP_STIPEND,
      levels = c(1,2),
      labels = c("Yes","No")), # No NAs
    RECD_TXT_BOOKS = factor(
```

```

    if_else(RECD_TXT_BOOKS %in% c(1,2), "Yes", "No"),
    levels = c("Yes", "No")), # 80 NAs, labeled No
  RECD_STATIONERY = factor(
    if_else(RECD_STATIONERY %in% c(1,2), "Yes", "No"),
    levels = c("Yes", "No")), # 151 NAs, labeled No
  MID_DAY_MEAL_ETC_RECD = factor(
    MID_DAY_MEAL_ETC_RECD,
    levels = c(1,2),
    labels = c("Yes", "No")), # No NAs
  .keep = "none"
)

# Merge with more data on those aged 5-19 who are in school
temp <- edu0708b6 %>%
  filter(AGE <= 19) %>%
  mutate(
    PID = PID,
    district_code = district_code,
    enrolled = 1,
    weight = weight,
    FSU_SL_NO = FSU_SL_NO,
    HHID = HHID,
    STATE = STATE,
    STRATUM = STRATUM,
    SUB_STRATUM_NO = SUB_STRATUM_NO
  ) %>%
  select(enrolled, weight, PID, district_code, TUTION_FEE, EXAMINATION_FEE,
    ↪ OTHER_FEES_PAYMENTS, BOOKS, STATIONERY, UNIFORM, TRANSPORT, FSU_SL_NO, HHID, STATE,
    ↪ STRATUM, SUB_STRATUM_NO)

selection_df <- full_join(selection_df, temp, by = join_by(PID, district_code, enrolled,
  ↪ weight, FSU_SL_NO, HHID, STATE, STRATUM, SUB_STRATUM_NO)) # Equivalent to excluding
  ↪ district_code, enrolled, and weight; doing this, however, prevents equivalent .x and
  ↪ .y columns from forming (e.g., enrolled.x and enrolled.y)

```

```

# Merge with data on all who are aged 5-19, including those who are not enrolled in
↳ school
temp <- edu0708b4 %>%
  filter(AGE >= 5, AGE <= 19) %>%
  mutate(
    AGE = AGE,
    SEX = factor(
      SEX,
      levels = c(1,2),
      labels = c("Male","Female")),
    HH_SIZE = HH_SIZE,
    RELIGION = factor(
      RELIGION,
      levels = 1:8,
      labels = c("Hindu","Muslim","Christian","Sikh","Jain","Buddhist","Zoroastrian","Other"
↳ her")),
    SOCIAL_GROUP = factor(
      SOCIAL_GROUP,
      levels = c(1,2,3,9),
      labels = c("Scheduled Tribe","Scheduled Caste","Other Backward Class","Other")),
    SECTOR = factor(
      SECTOR,
      levels = c(1, 2),
      labels = c("Rural","Urban")),
    DIST_FROM_NEAREST_PRIMARY_CLASS = factor(
      DIST_FROM_NEAREST_PRIMARY_CLASS,
      levels = 1:5,
      labels = c("d<1km",
        "1km <= d <2kms",
        "2kms<= d <3kms",

```

```

        "3kms <= d <5kms",
        "d>=5kms")),
    district_code = district_code,
    PID = PID,
    weight = weight,
    FSU_SL_NO = FSU_SL_NO,
    HHID = HHID,
    STATE = STATE,
    STRATUM = STRATUM,
    SUB_STRATUM_NO = SUB_STRATUM_NO,
    .keep = "none"
)

selection_df <- full_join(selection_df, temp, by = join_by(PID, district_code, weight,
  ↳ FSU_SL_NO, HHID, STATE, STRATUM, SUB_STRATUM_NO))

#### Father's education proxy
# Merge with data on father's education
# @azam2013a, @munshi2006a, @singh2013: Father's education as proxy of ability. So use
  ↳ father's education for father_educ

# Father proxy: within-HH male and potential parent. Defined for each HHID in this
  ↳ priority order:
# Male head (RELATION_TO_HEAD==1 & SEX==1)
# Male married child of head (3 & 1) (common for married male to live with parents)
# Male spouse of head (2 & 1) ***OR MAYBE 5 & 1!!! CHANGE IN FUTURE. Misclassification
  ↳ error in data here.
# Male parent/parent-in-law of head (7 & 1)

```

```

# Build a household-level "father proxy" table
father_proxy <- edu0708b4 %>%

  mutate(
    educ_collapsed = fct_collapse(
      EDUCATION_LEVEL,
      Illiterate = "01",
      `Literate, no school` = c("02","03","04","05"),
      `Literate, school < primary` = "06",
      Primary = "07",
      `Upper primary` = "08",
      Secondary = "10",
      `Higher secondary` = c("11","12"),
      `Postsecondary+` = c("13","14"),
      .default = NA_character_
    )
  ) %>%
  mutate(
    # priority rank (lower = better)
    father_rank = case_when(
      RELATION_TO_HEAD == 1 & SEX == 1 ~ 1L, # Male head
      RELATION_TO_HEAD == 3 & SEX == 1 ~ 2L, # Male married child
      # RELATION_TO_HEAD == 2 & SEX == 1 ~ 3L, # Male spouse of head; average age of this
      # → 11.3, or 11.0 after filtering for female heads. Misclassification error!
      RELATION_TO_HEAD == 7 & SEX == 1 ~ 4L, # Male parent/parent-in-law of head
      TRUE ~ NA_integer_ # Have to specify all are ints
    )
  ) %>%
  group_by(HHID) %>%
  arrange(father_rank, .by_group = TRUE) %>%
  slice(1L) %>% # Best available candidate
  ungroup() %>%
  mutate(

```

```

HHID,
father_educ = factor(educ_collapsed,
                      levels = c("Illiterate", "Literate, no school", "Literate,
                                ↪ school < primary", "Primary", "Upper
                                ↪ primary", "Secondary", "Higher
                                ↪ secondary", "Postsecondary+")),
                      .keep = "none"
)

# Join to selection_df (adds the same parental_educ to all kids in HH)
selection_df <- selection_df %>% left_join(father_proxy, by = "HHID")

#### Enrollment and NA cost removal

# Create "enrolled" variable
selection_df <- selection_df %>%
  rename(district_code_0708 = district_code) %>%
  mutate(
    # any child not in edu0708b5/b6 gets enrolled=0
    enrolled = if_else(is.na(enrolled), 0, enrolled),
    enrolled = factor(enrolled, levels = 0:1, labels = c("No", "Yes")),
    district_code_0708 = as.factor(district_code_0708)
  )

# Create enrollment cost.
# Use documentation to remove some NAs (else many NAs would come from e.g., kids with
  ↪ RECD_TXT_BOOKS=="Yes" but an NA in BOOKS)
selection_df <- selection_df %>%

```

```

mutate(
  # Remove some NAs: Either condition below implies free tuition, per NSS documentation
  TUTION_FEE = if_else(
    is.na(TUTION_FEE) & (IS_EDU_FREE == "Yes" | TUTION_FEE_WAIVED == "Yes"),
    0, TUTION_FEE
  ),
  # Textbooks received -> books cost 0 if missing
  BOOKS = if_else(
    is.na(BOOKS) & RECD_TXT_BOOKS == "Yes",
    0, BOOKS
  ),
  # Stationery received -> stationery cost 0 if missing
  STATIONERY = if_else(
    is.na(STATIONERY) & RECD_STATIONERY == "Yes",
    0, STATIONERY
  ),
  ENROLLMENT_COST = TUTION_FEE + # Same spelling as in data
    EXAMINATION_FEE +
    OTHER_FEES_PAYMENTS +
    BOOKS +
    STATIONERY +
    UNIFORM +
    TRANSPORT
)

# Explicitly set reference level for all factors
# (Note that the 1st level when defining the variables is by default the reference e.g.,
#   ↪ "Hindu" was already the reference level for RELIGION):
selection_df <- selection_df %>%
  mutate(

```

```

RELIGION = relevel(RELIGION, ref = "Hindu"),
SOCIAL_GROUP = relevel(SOCIAL_GROUP, ref = "Other"),
DIST_FROM_NEAREST_PRIMARY_CLASS =
  relevel(DIST_FROM_NEAREST_PRIMARY_CLASS, ref = "d<1km"),
father_educ = relevel(father_educ, ref = "Illiterate")
)

# Confirm
# levels(selection_df$RELIGION)
# levels(selection_df$SOCIAL_GROUP)
# levels(selection_df$DIST_FROM_NEAREST_PRIMARY_CLASS)
# levels(selection_df$father_educ)

#### District-level aggregates for imputation ####

# Many relevant variables are only defined for enrolled kids. In lieu of using multiple
→ imputation to assign them for not-enrolled kids, we use district-level aggregates of
→ these variables as instruments for each child in the district. The exclusion
→ restriction of the Heckman correction holds for the aggregates as it does for the
→ original variables.

enrolled_only_vars <- c(
  "IS_EDU_FREE", "TUTION_FEE_WAIVED", "RECD_SCHOLARSHIP_STIPEND",
  "RECD_TXT_BOOKS", "RECD_STATIONERY", "MID_DAY_MEAL_ETC_RECD",
  "ENROLLMENT_COST"
)

# Ensure binary factors, ENROLLMENT_COST are numeric
temp <- selection_df %>%
  mutate(
    across(all_of(enrolled_only_vars),

```



```

    ~case_when(
      # for factors that are Yes/No:
      is.factor(.x) ~ as.numeric(.x == "Yes"),
      # for cost, should already be numeric:
      TRUE ~ as.numeric(.x)
    ),
    .names="num_{col}")
)

# Take their weighted district-level means
district_agg <- temp %>%
  filter(enrolled == "Yes") %>%
  group_by(district_code_0708) %>%
  summarize(across(starts_with("num_"), # the numeric versions
    ~ {
      val <- weighted.mean(.x, w=weight, na.rm=TRUE)
      if (is.nan(val)) NA_real_ else val # Convert NaN (from dividing by
        ↪ 0, no one in district has it) to NA
    },
    .names = "dmean_{.col}"),
    .groups="drop"
  )

# For future: Use regional and state level if district level does not exist

# Bring those means in without overwriting
selection_df <- selection_df %>%
  left_join(district_agg, by="district_code_0708")

# May be better to give unenrolled kids the weighted average enrollment cost of enrolled
↪ kids who are a) in the same district, b) with the same values for all the
↪ probit_controls variables, except for c) AGE, where we include kids up to two years
↪ younger and two years older? May end up with far more missing values, however.

```

```

# Merge state names, region names
selection_df <- selection_df %>% left_join(
  districts_0708 %>% mutate(district_code_0708 = as.character(district_code_0708)),
  by = "district_code_0708")

# Build regressor lists
probit_controls <- c("AGE",
                    "SEX",
                    "HH_SIZE",
                    "RELIGION",
                    "SOCIAL_GROUP",
                    "SECTOR")

probit_excl_restr_all_kids <- c("DIST_FROM_NEAREST_PRIMARY_CLASS",
                              "father_educ")

probit_excl_restr_enrolled_only <- c(
  "dmean_num_IS_EDU_FREE", "dmean_num_TUTION_FEE_WAIVED", "dmean_num_RECD_SCHOLARSHIP_STIP",
  ↪ END",
  "dmean_num_RECD_TXT_BOOKS", "dmean_num_RECD_STATIONERY", "dmean_num_MID_DAY_MEAL_ETC_REC",
  ↪ D",
  "dmean_num_ENROLLMENT_COST"
)

probit_vars <- c(probit_controls, probit_excl_restr_all_kids,
  ↪ probit_excl_restr_enrolled_only)

# Create formula from regressor vectors

```

```
f_probit <- reformulate(
  probit_vars,
  response = "enrolled"
)
```

Chunk 8: Participation model: Is missingness associated with observed covariates

```
#### All variables: Variables/Regions with NAs ####

# Variables with NAs
# Only variables in the probit
counts <- selection_df %>%
  summarise(across(all_of(probit_vars), ~ sum(is.na(.)))) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "NA_count") %>%
  filter(NA_count >= 0) %>%
  arrange(desc(NA_count))

total <- selection_df %>%
  mutate(any_NA_probit = !complete.cases(across(all_of(probit_vars)))) %>%
  summarise(n = sum(any_NA_probit)) %>%
  pull(n)

sel_NA_vars <- bind_rows(
  counts,
  tribble(~variable, ~NA_count,
    "Total probit-relevant with NA", total,
    "Total probit-relevant with no NA", selection_df %>%
      select(all_of(probit_vars)) %>%
      drop_na() %>% nrow())
)
```

```

)

# Regions with the _most_ NAs

temp <- selection_df %>%

  mutate(
    any_na_row = !complete.cases(across(all_of(probit_vars))), # ANY NA in the whole row
    ↪ (strict)
    across(
      all_of(
        c("dmean_num_ENROLLMENT_COST",
          "father_educ",
          "DIST_FROM_NEAREST_PRIMARY_CLASS"
        )
      ),
      ~ as.integer(is.na(.)),
      .names = "miss_{col}"),
    # Demographic shares
    is_urban = as.integer(SECTOR == "Urban"),
    is_female = as.integer(SEX == "Female"),
    is_hindu = as.integer(RELIGION == "Hindu"),
    is_muslim = as.integer(RELIGION == "Muslim"),
    is_st_sc_obc = as.integer(SOCIAL_GROUP %in% c("Scheduled Tribe", "Scheduled
    ↪ Caste", "Other Backward Class"))
  )

sel_NA_regions_cost <- temp %>%
  group_by(state_0708, region_0708) %>%
  summarise(
    n = n(),
    pct_any_na = mean(any_na_row, na.rm = TRUE),

```

```

pct_miss_dmean_num_ENROLLMENT_COST = mean(miss_dmean_num_ENROLLMENT_COST, na.rm =
  ↪ TRUE),
pct_miss_DIST_FROM_NEAREST_PRIMARY = mean(miss_DIST_FROM_NEAREST_PRIMARY_CLASS, na.rm
  ↪ = TRUE),
pct_miss_father_educ = mean(miss_father_educ, na.rm = TRUE),
pct_urban = mean(is_urban, na.rm = TRUE),
pct_female = mean(is_female, na.rm = TRUE),
pct_hindu = mean(is_hindu, na.rm = TRUE),
pct_muslim = mean(is_muslim, na.rm = TRUE),
pct_st_sc_obc = mean(is_st_sc_obc, na.rm = TRUE),
.groups = "drop"
) %>%
arrange(desc(pct_miss_dmean_num_ENROLLMENT_COST)) %>%
slice_head(n = 20) %>%
mutate(across(starts_with("pct_"), ~round(100*.x, 2))) # to percentages

# Case study:
# selection_df %>%
#   filter(
#     state_0708=="Rajasthan" &
#     region_0708=="Southern"
#   ) %>%
#   filter(if_any(everything(), is.na)) %>%
#   select(where(anyNA)) %>%
#   summarise(across(everything(), ~sum(is.na(.)))) %>%
#   View
# selection_df %>%
#   filter(
#     state_0708=="Rajasthan" &
#     region_0708=="Southern"
#   ) %>%
#   filter(if_any(everything(), is.na)) %>%

```

```

# select(where(anyNA)) %>%
# View

# Southern region of Rajasthan: People with one missing cost variable often have ones
  ↳ which aren't missing. The person as a whole wasn't excluded from the data.
# Explanations: Surveyor messed up writing some costs but not others. Or surveyor meant
  ↳ to put a 0.

sel_NA_regions_dist <- temp %>%
  group_by(state_0708, region_0708) %>%
  summarise(
    n = n(),
    pct_any_na = mean(any_na_row, na.rm = TRUE),
    pct_miss_dmean_num_ENROLLMENT_COST = mean(miss_dmean_num_ENROLLMENT_COST, na.rm =
      ↳ TRUE),
    pct_miss_DIST_FROM_NEAREST_PRIMARY = mean(miss_DIST_FROM_NEAREST_PRIMARY_CLASS, na.rm
      ↳ = TRUE),
    pct_miss_father_educ = mean(miss_father_educ, na.rm = TRUE),
    pct_urban = mean(is_urban, na.rm = TRUE),
    pct_female = mean(is_female, na.rm = TRUE),
    pct_hindu = mean(is_hindu, na.rm = TRUE),
    pct_muslim = mean(is_muslim, na.rm = TRUE),
    pct_st_sc_obc = mean(is_st_sc_obc, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(desc(pct_miss_DIST_FROM_NEAREST_PRIMARY)) %>%
  slice_head(n = 20) %>%
  mutate(across(starts_with("pct_"), ~round(100*.x, 2)))

sel_NA_regions_father <- temp %>%
  group_by(state_0708, region_0708) %>%
  summarise(
    n = n(),

```

```

pct_any_na = mean(any_na_row, na.rm = TRUE),
pct_miss_dmean_num_ENROLLMENT_COST = mean(miss_dmean_num_ENROLLMENT_COST, na.rm =
↪ TRUE),
pct_miss_DIST_FROM_NEAREST_PRIMARY = mean(miss_DIST_FROM_NEAREST_PRIMARY_CLASS, na.rm
↪ = TRUE),
pct_miss_father_educ = mean(miss_father_educ, na.rm = TRUE),
pct_urban = mean(is_urban, na.rm = TRUE),
pct_female = mean(is_female, na.rm = TRUE),
pct_hindu = mean(is_hindu, na.rm = TRUE),
pct_muslim = mean(is_muslim, na.rm = TRUE),
pct_st_sc_obc = mean(is_st_sc_obc, na.rm = TRUE),
.groups = "drop"
) %>%
arrange(desc(pct_miss_father_educ)) %>%
slice_head(n = 20) %>%
mutate(across(starts_with("pct_"), ~round(100*.x, 2)))

# # Potential chi-square tests for independence
# # ANY NA vs. state
# chisq.test(table(temp$any_na_row, temp$state_0708))
# # ANY NA vs region
# chisq.test(table(temp$any_na_row, temp$region_0708))

# Define all variables

# Missing variables defined only for enrolled
miss_vars_enrolled <- c("TUTION_FEE", "EXAMINATION_FEE", "OTHER_FEES_PAYMENTS",
↪ "BOOKS", "STATIONERY", "UNIFORM", "TRANSPORT") # ENROLLMENT_COST excluded

```

```

# Missing variables defined for all
miss_vars_all <- c("DIST_FROM_NEAREST_PRIMARY_CLASS",
                  "dmean_num_ENROLLMENT_COST",
                  "father_educ")

miss_vars <- c(miss_vars_all, miss_vars_enrolled)

# Variables I'm most worried about explaining these misses
group_vars <- c("SECTOR", "SEX", "RELIGION", "SOCIAL_GROUP", "state_0708")
cts_vars <- c("AGE", "HH_SIZE")

#### Correlation matrix ####

# First, for all observations
# Make new df with missingness indicator vars
temp1 <- selection_df %>%
  mutate(
    across(all_of(miss_vars_all),
           ~as.integer(is.na(.)),
           .names = "miss_{.col}"),
    .keep = "none"
  )

# Expand factors to set of dummy variables
temp2 <- model.matrix(~.-1, data = selection_df %>% select(all_of(group_vars))) # -1
  ↪ removes intercept column

# Compute correlation matrix
sel_corr_mat_all <- cor(
  cbind(

```



```

    temp1,
    selection_df %>% select(all_of(cts_vars)),
    temp2
  ),
  use = "pairwise.complete.obs" # Corr found for complete pairs. NA entries allowed.
  ↪ Pearson method.
)

# Second, only for enrolled observations
# Make new df with missingness indicator vars
temp1 <- selection_df %>%
  filter(enrolled=="Yes") %>%
  mutate(
    across(all_of(miss_vars_enrolled),
      ~as.integer(is.na(.)),
      .names = "miss_{.col}"),
    .keep = "none"
  )

# Expand factors to set of dummy variables
temp2 <- model.matrix(~.-1,
  data = selection_df %>% filter(enrolled=="Yes") %>%
    ↪ select(all_of(group_vars)),
  na.action=na.pass
)

# Compute correlation matrix
sel_corr_mat_enrolled <- cor(
  cbind(
    temp1,
    selection_df %>% filter(enrolled=="Yes") %>% select(all_of(cts_vars)),
    temp2
  ),
  use = "pairwise.complete.obs" # Corr found for complete pairs. Allows for NA entries.
  ↪ Pearson method.

```

```

)

#### Logistic per missing variable ####

check_missing_logit_parallel <- function(df, miss_vars, covars, method_p = "BH") {
  # RHS of regression
  rhs <- paste(covars, collapse = " + ")

  # To fit the binary outcome is.na(m) for each missing var m
  fit_one <- function(m) {
    f <- as.formula(paste0("is.na(", m, ") ~ ", rhs))
    fit <- glm(f, data = df, family = binomial)
    pseudoR2 <- 1 - fit$deviance / fit$null.deviance # Note: No missingness indicator was
    ↪ constant
    gl <- glance(fit) # Currently unused; retained for future diagnostics

    # Return tidy df of coefficients and attach missing var's name
    broom::tidy(fit) %>% # Namespace made explicit for cluster workers
      mutate(
        missing_var = m,
        pseudoR2 = pseudoR2,
        nobs = stats::nobs(fit)
      )
  }

  # Parallelization
  # Cluster with parLapply() if on Windows
  out_list <- if (.Platform$OS.type == "windows") {
    cl <- parallel::makeCluster(max(1, parallel::detectCores() - 1))
    on.exit(parallel::stopCluster(cl), add = TRUE) # Shut down cluster even if errors
    ↪ occur
  }

```

```

parallel::clusterEvalQ(cl, {
  library(broom)
  library(dplyr)
})
parallel::clusterExport(
  cl,
  varlist = c("df", "rhs", "fit_one"),
  envir = environment()
)
parallel::parLapply(cl, miss_vars, fit_one)
} else { # Fork with mclapply()
  parallel::mclapply(miss_vars, fit_one, mc.cores = max(1, parallel::detectCores() -
↪ 1))
} # Returns list of tidy dfs, one per missing var

out <- bind_rows(out_list) # Stack the dfs
out <- out %>%
  group_by(missing_var) %>%
  mutate(
    p_adj = { # Adjust within each missingness model
      adj <- rep(NA_real_, dplyr::n())
      idx <- term != "(Intercept)" # Exclude intercept
      adj[idx] <- p.adjust(p.value[idx], method = method_p)
      adj
    }
  ) %>%
  ungroup()
out
}

covars <- c("SECTOR", "SEX", "AGE", "HH_SIZE", "RELIGION", "SOCIAL_GROUP", "state_0708")

# Run on missing variables defined for all children

```

```

logit_missing_results_all <- check_missing_logit_parallel(selection_df, miss_vars_all,
  ↪ covars, method_p = "BH") # "BH" = Benjamini-Hochberg control on false discovery rate.
  ↪
# "BY" lets independence or positive dependence assumptions weaken to arbitrary
  ↪ dependence, but has weaker power (i.e., more p-vals above 0.05).

sel_logit_NA_all <- logit_missing_results_all %>%
  group_by(missing_var) %>% # Use name from fit_one()
  summarise(
    n_sig = sum(p_adj < 0.05 & term != "(Intercept)",
    pseudoR2 = max(pseudoR2, na.rm = TRUE) # Model-level metric
  ) %>%
  arrange(desc(pseudoR2))
# So n_sig = # of predictors which survive FDR at 0.05 significance (i.e., # of
  ↪ covariates associated with missingness)

# Predictors of missingness for missing vars in probit (dmean_num_ENROLLMENT_COST and
  ↪ DIST_FROM_NEAREST_PRIMARY_CLASS)
sel_predlogit_NA_all <- logit_missing_results_all %>%
  filter(
    missing_var %in% miss_vars,
    term != "(Intercept)",
    p_adj < 0.05) %>%
  mutate(predictor = term) %>%
  select(missing_var, predictor, estimate, p_adj) %>%
  arrange(missing_var, p_adj, desc(abs(estimate)))

# Plot results: variables with most non-random missingness
plt_sel_logit_NA_all <- logit_missing_results_all %>%
  group_by(missing_var) %>%
  summarise(pseudoR2 = max(pseudoR2, na.rm = TRUE)) %>%
  ggplot(aes(reorder(missing_var, pseudoR2), pseudoR2)) +

```

```

geom_col() +
coord_flip() +
labs(y = "Pseudo-R2 (predictability of missingness)",
      x = "Variable",
      title = "How structured is missingness?")

# Run on missing variables defined for schoolgoing children only
logit_missing_results_enrolled <- check_missing_logit_parallel(selection_df,
  ↪ miss_vars_enrolled, covars, method_p = "BH") # "BH" = Benjamini-Hochberg control on
  ↪ false discovery rate.
# "BY" lets independence or positive dependence assumptions weaken to arbitrary
  ↪ dependence, but has weaker power (i.e., more p-vals above 0.05).

sel_logit_NA_enrolled <- logit_missing_results_enrolled %>%
  group_by(missing_var) %>% # Use name from fit_one()
  summarise(
    n_sig = sum(p_adj < 0.05 & term != "(Intercept)"),
    pseudoR2 = max(pseudoR2, na.rm = TRUE) # Model-level metric
  ) %>%
  arrange(desc(pseudoR2))
# So n_sig = # of predictors which survive FDR at 0.05 significance (i.e., # of
  ↪ covariates associated with missingness)

# Predictors of missingness for missing vars in probit (dmean_num_ENROLLMENT_COST and
  ↪ DIST_FROM_NEAREST_PRIMARY_CLASS)
sel_predlogit_NA_enrolled <- logit_missing_results_enrolled %>%
  filter(
    missing_var %in% miss_vars,
    term != "(Intercept)",
    p_adj < 0.05) %>%
  mutate(predictor = term) %>%
  select(missing_var, predictor, estimate, p_adj) %>%

```

```

arrange(missing_var, p_adj, desc(abs(estimate)))

# Plot results: variables with most non-random missingness
plt_sel_logit_NA_enrolled <- logit_missing_results_enrolled %>%
  group_by(missing_var) %>%
  summarise(pseudoR2 = max(pseudoR2, na.rm = TRUE)) %>%
  ggplot(aes(reorder(missing_var, pseudoR2), pseudoR2)) +
  geom_col() +
  coord_flip() +
  labs(y = "Pseudo-R2 (predictability of missingness)",
       x = "Variable",
       title = "How structured is missingness?")

# Potential tests for later: Multivariate logistic regression; variable-by-variable
→ chi-squared, t-tests

```

Chunk 9: Participation model: Estimation and IMR

```

# Handling lonely PSUs ("variance formula gives 0/0 if stratum contains only one
→ sampling unit", `?svyCprod`)

# Ex: The "Jhelam Valley" stratum has only one PSU in the data.

# Options: "'fail'" to give an error, "remove" or "certainty" to give a variance
→ contribution of 0 for the stratum, "adjust" to center the stratum at the grand mean
→ rather than the stratum mean, and "average" to assign strata with one PSU the average
→ variance contribution from strata with more than one PSU."

# "'adjust'" is conservative, and it would often be better to combine strata in some
→ intelligent way."

# "properties of 'average' have not been investigated thoroughly, but it may be useful
→ when the lonely PSUs are due to a few strata having PSUs missing completely at
→ random."

```

```

# "'remove' and 'certainty' options give the same result"
# "but 'certainty' is intended for situations where there is only one PSU in the
  ↳ population stratum, which is sampled with certainty (also called 'self-representing'
  ↳ PSUs or strata). With 'certainty' no warning is generated for strata with only one
  ↳ PSU. Ordinarily, svydesign will detect certainty PSUs, making this option
  ↳ unnecessary."

# Will thus use "average" for handling single-PSU strata (e.g., "Jhelam Valley") when
  ↳ estimating variance
options(survey.lonely.psu = "average")

# Create weighted survey design model
selection_design <- svydesign(

  ids = ~FSU_SL_NO + HHID,

  # `?svydesign`: "For multistage sampling the id argument should specify a formula with
    ↳ the cluster identifiers at each stage."

  # Sampling occurred in two stages: the first-stage units (FSU) were rural
    ↳ villages/urban blocks (FSU_SL_NO), and the ultimate stage units (USUs) were
    ↳ households (HHID)

  strata = ~interaction(
    as.character(STATE),
    as.character(STRATUM),
    as.character(SUB_STRATUM_NO),
    drop = TRUE # Nothing dropped yet
  ),

  # Sampling stratified into state, stratum (district), and substratum (rural/urban, with
    ↳ multiple urban substrata if population dense enough)

  weights = ~weight,
  data = selection_df,

```

```

    nest = TRUE # Clusters (PSUs) are nested within strata (states*strata*substrata)
)

# Estimate weighted model
model_probit_selection <- svyglm(
  f_probit,
  design = selection_design,
  family = binomial(link = "probit")
) # Rows with an NA for a probit-relevant variable dropped!

# Derive IMR
# For now, calculate IMR for all rows. Don't drop rows with NAs in unrelated columns:

# Rows with no NA under probit-relevant variable
needed_vars <- all.vars(f_probit) # = c(probit_vars, "enrolled")

# Take only complete cases of variables in f_probit
comp_cases <- complete.cases(selection_df[, needed_vars, drop = FALSE])

# Predict only on complete rows; NA for rest
lp <- rep(NA_real_, nrow(selection_df)) # full-length placeholders
lp[comp_cases] <- predict(model_probit_selection,
  newdata = selection_df[comp_cases, , drop=FALSE],
  type = "link") # to return linear predictors

# Build IMR
phi_lp <- dnorm(lp)
Phi_lp <- pnorm(lp)

```



```

# Remaining NAs could be from dividing by 0.
# Ensure against this by keeping Phi_lp \in (0,1) strictly
Phi_lp <- pmin(pmax(Phi_lp, .Machine$double.eps), 1 - .Machine$double.eps)
# .Machine$double.eps = 2.220446e-16 on my device

selection_df <- selection_df %>%

  mutate(
    IMR = case_when(
      enrolled == "Yes" ~ phi_lp / Phi_lp,
      enrolled == "No" ~ -phi_lp / (1 - Phi_lp),
      TRUE ~ NA_real_ # NA if missing predictor
    )
  )

```

Chunk 10: Participation model: Derive AME

```

# Use automatic differentiation (AD) for SE delta method gradients
autodiff(autodiff = TRUE, install = FALSE) # install=TRUE if self-managing Python install
options(marginaleffects_parallel = FALSE) # Make sure it's off
sel_data <- selection_df[comp_cases, , drop = FALSE]

num_samp <- 200
set.seed(999)
final_draft <- TRUE # Set TRUE when rendering final draft
newdata_ame <- if (final_draft) sel_data else sel_data %>% slice_sample(n = num_samp)

# start.time <- Sys.time()
mfx_all <- avg_slopes(
  model_probit_selection,
  newdata = newdata_ame,
  wts = "weight",

```

```

vcov = TRUE, # Set FALSE to make this even faster
type = "response"
)

# end.time <- Sys.time()
# tdiff <- end.time - start.time
# # With newdata = sel_data (no subsampling, as if num_samp = 279180): 6.178544-11.17861
#   ↪ mins when ran for the first time, 4.853338 mins later.
# # With num_samp = 20000: 36.13833 secs
# # With num_samp = 2000: 4.005744 secs

# The rest of this chunk is saved for quick replications of robustness checks on my final
#   ↪ method

# Raw AME derivation is very slow:
# 9 numeric variables * 279180 observations * 2 calls of predict() per observation per
#   ↪ variable (avg_slopes() uses centered finite difference, see https://marginaleffects.com/bonus/uncertainty.html#numerical-derivatives-sensitivity-to-step-size) = 5025240
#   ↪ predict() calls.

# Method 1: Straight Sampling and Subsampling
# Note: "slopes() functions will automatically revert to comparisons() for binary or
#   ↪ categorical variables" (https://marginaleffects.com/man/r/slopes.html)

# num_samp <- 2000
# set.seed(999)
# newdata_ame <- sel_data %>% slice_sample(n = num_samp)
# # newdata_ame <- sel_data
#
# start.time <- Sys.time()

```

```

#
# mfx_all <- avg_slopes(
#   model_probit_selection,
#   newdata = newdata_ame,
#   wts = "weight",
#   vcov = TRUE # Set as FALSE to make this even faster
# )
#
# end.time <- Sys.time()
# end.time - start.time

# With newdata = sel_data (no subsampling, as if num_samp = 279180): ~40 minutes!
# With num_samp = 20000: 36.7708 secs
# With num_samp = 2000: 4.429373 secs

# Method 2: Subsampling with forward differences
# Note: Calls predict() once per perturbation instead of twice (a la central differences,
#   ↪ "fdcenter")
# Run the above, but add numberiv = "fdforward" into avg_slopes()
# With num_samp = 20000: 37.41933 secs
# With num_samp = 2000: 4.047509 secs
# Cheaper numeric differentiation didn't help at all! Reflective of how many discrete
#   ↪ variables I have.
# Centered differences are generally more accurate than forward differences for
#   ↪ continuous vars

# Method 3: Split slopes and comparisons, just to be sure
# Ran avg_slopes on numeric_vars, avg_comparisons on factor_vars
# With num_samp = 20000: 38.94813 secs
# With num_samp = 2000: 5.039992 secs
# No. Just use avg_slopes()

```

```

# Method 4: Parallelization

# library(future.apply) # For parallelization
# #
# # Resolve futures in parallel in *forked* R processes. NOT SUPPORTED ON WINDOWS!
# plan(multicore, workers = 4) # Using "multisession" led to the same result.
#
# options(marginaleffects_parallel = TRUE) # parallelize delta method computation of
  ↪ standard errors
#
# num_samp = 2000
# set.seed(999)
# newdata_ame <- sel_data %>% slice_sample(n = num_samp)
#
# start.time <- Sys.time()
#
# mfx_all <- avg_slopes(
#   model_probit_selection,
#   newdata = newdata_ame,
#   wts = "weight",
#   vcov = TRUE # Set as FALSE to make this even faster
# )
#
# end.time <- Sys.time()
# end.time - start.time

# Failed! "The total size of the 18 globals exported for future expression ('FUN()') is
  ↪ 7.90 GiB." Three largest globals: "'FUN' (3.54 GiB of class 'function'), 'func' (3.54
  ↪ GiB of class 'function') and 'mfx' (544.70 MiB of class 'S4')."

# As documentation states: "There is always considerable overhead when using parallel
  ↪ computation, mainly involved in passing the whole dataset to the different
  ↪ processes." (https://cloud.r-project.org/web/packages/marginaleffects/refman/marginaleffects.html)

```

```

# To parallelize for the whole dataset after running `environment(model_probit_selection)
↪  <- NULL; model_probit_selection$survey.design <- NULL; model_probit_selection$data <-
↪  NULL`:
# 53.96 GiB. "The three largest globals are 'FUN' (24.00 GiB of class 'function'), 'func'
↪  (24.00 GiB of class 'function') and 'hi' (1.82 GiB of class 'list')"
```

My laptop has 24 GB of RAM! So no parallelization.

Chunk 11: Participation model: Set up results table

```

# Define covariate labels, same order in coef_me, se_me, p_me:
labels <- c(
  "Age (years)",
  "Female (ref: Male)",
  "Household size",
  # Religion dummies (ref: Hindu)
  "Religion: Muslim (ref: Hindu)",
  "Religion: Christian",
  "Religion: Sikh",
  "Religion: Jain",
  "Religion: Buddhist",
  "Religion: Zoroastrian",
  "Religion: Other",
  # Social group dummies (ref: Other)
  "Social group: Scheduled Tribe (ref: Other)",
  "Social group: Scheduled Caste",
  "Social group: Other Backward Class",
  # sector
  "Urban (ref: Rural)",
```

```

# distance dummies (ref: <1km)
"Distance 1-2km (ref: <1km)",
"Distance 2-3km",
"Distance 3-5km",
"Distance > 5km",
# parental education levels (ref: Illiterate)
"Father's educ.: Literate, no school (ref: Illiterate)",
"Father's educ.: Literate, school < primary",
"Father's educ.: Primary",
"Father's educ.: Upper primary",
"Father's educ.: Secondary",
"Father's educ.: Higher secondary",
"Father's educ.: Postsecondary+",
# district-level instruments (shares; ref = 0)
"Educ. free available (ref: No)",
"Tuition waiver received",
"Scholarship/Stipend received",
"Textbook(s) received",
"Stationery received",
"Mid-day meal, etc. received",
# mean cost
"Enrollment cost (Rs.)"
)

# Map (term, contrast) in mfx_all to label in labels
lookup <- tribble(
  ~term, ~contrast, ~Term,
  "AGE", "dY/dX", "Age (years)",
  "SEX", "Female - Male", "Female (ref: Male)",
  "HH_SIZE", "dY/dX", "Household size",

```

```

# RELIGION (ref: Hindu)

"RELIGION","Muslim - Hindu","Religion: Muslim (ref: Hindu)",
"RELIGION","Christian - Hindu","Religion: Christian",
"RELIGION","Sikh - Hindu","Religion: Sikh",
"RELIGION","Jain - Hindu","Religion: Jain",
"RELIGION","Buddhist - Hindu","Religion: Buddhist",
"RELIGION","Zoroastrian - Hindu","Religion: Zoroastrian",
"RELIGION","Other - Hindu","Religion: Other",

# SOCIAL_GROUP (ref: Other) - match labels exactly

"SOCIAL_GROUP","Scheduled Tribe - Other","Social group: Scheduled Tribe (ref: Other)",
"SOCIAL_GROUP","Scheduled Caste - Other","Social group: Scheduled Caste",
"SOCIAL_GROUP","Other Backward Class - Other","Social group: Other Backward Class",

# SECTOR

"SECTOR","Urban - Rural","Urban (ref: Rural)",

# DIST_FROM_NEAREST_PRIMARY_CLASS (ref: d<1km)

"DIST_FROM_NEAREST_PRIMARY_CLASS","1km <= d <2kms - d<1km","Distance 1-2km (ref:
↳ <1km)",
"DIST_FROM_NEAREST_PRIMARY_CLASS","2kms<= d <3kms - d<1km","Distance 2-3km",
"DIST_FROM_NEAREST_PRIMARY_CLASS","3kms <= d <5kms - d<1km","Distance 3-5km",
"DIST_FROM_NEAREST_PRIMARY_CLASS","d>=5kms - d<1km","Distance > 5km",

# father_educ (ref: Illiterate)

"father_educ","Literate, no school - Illiterate","Father's educ.: Literate, no school
↳ (ref: Illiterate)",
"father_educ","Literate, school < primary - Illiterate","Father's educ.: Literate,
↳ school < primary",
"father_educ","Primary - Illiterate","Father's educ.: Primary",
"father_educ","Upper primary - Illiterate","Father's educ.: Upper primary",

```

```

"father_educ","Secondary - Illiterate","Father's educ.: Secondary",
"father_educ","Higher secondary - Illiterate","Father's educ.: Higher secondary",
"father_educ","Postsecondary+ - Illiterate","Father's educ.: Postsecondary+",

# district-level means
"dmean_num_IS_EDU_FREE","dY/dX","Educ. free available (ref: No)",
"dmean_num_TUTION_FEE_WAIVED","dY/dX","Tuition waiver received",
"dmean_num_RECD_SCHOLARSHIP_STIPEND","dY/dX","Scholarship/Stipend received",
"dmean_num_RECD_TXT_BOOKS","dY/dX","Textbook(s) received",
"dmean_num_RECD_STATIONERY","dY/dX","Stationery received",
"dmean_num_MID_DAY_MEAL_ETC_RECD","dY/dX","Mid-day meal, etc. received",
"dmean_num_ENROLLMENT_COST","dY/dX","Enrollment cost (Rs.)"
)

# Make sure it mapped correctly
mfx_df <- as.data.frame(mfx_all) %>%
  dplyr::left_join(lookup, by = c("term","contrast"))

# Catch unmapped rows
unmapped <- mfx_df %>% dplyr::filter(is.na(Term)) %>% dplyr::select(term, contrast) %>%
  ↪ unique()
if (nrow(unmapped) > 0) {
  warning("Unmapped (term, contrast) pairs found:\n",
    paste0(utils::capture.output(print(unmapped)), collapse = "\n"))
}

# Build df and add stars & parentheses:

```



```

reg_tbl_selection <- as.data.frame(mfx_df) %>%
  mutate(
    stars = case_when(
      p.value < 0.001 ~ "***",
      p.value < 0.01 ~ "**",
      p.value < 0.05 ~ "*",
      TRUE ~ ""
    ),
    Estimate = sprintf("%.3f%s", estimate, stars),
    `Std. Error` = sprintf("(%.3f)", std.error),
    Term = factor(Term, levels = labels)
  ) %>%
  arrange(Term) %>%
  select(Term, Estimate, `Std. Error`)

```

Chunk 12: Participation model: Make summary tables

```

## selection_df quant metadata:
sel_quant_meta <- tribble(
  ~var, ~label,
  "AGE", "Age",
  "HH_SIZE", "Household size",
  "ENROLLMENT_COST", "Enrollment cost (Rs.)",
  # district-level:
  "dmean_num_IS_EDU_FREE", "Educ. free available? (Yes = 1)",
  "dmean_num_TUTION_FEE_WAIVED", "Tuition waived?",
  "dmean_num_RECD_SCHOLARSHIP_STIPEND", "Scholarship/Stipend?",
  "dmean_num_RECD_TXT_BOOKS", "Textbooks received?",
  "dmean_num_RECD_STATIONERY", "Stationery received?",
  "dmean_num_MID_DAY_MEAL_ETC_RECD", "Mid-day meal or more received?"
)

```

```

## selection_df categorical metadata:
sel_cat_meta <- tribble(
  ~var, ~label,
  "SEX", "Sex",
  "RELIGION", "Religion",
  "SOCIAL_GROUP", "Social group",
  "SECTOR", "Urban",
  "DIST_FROM_NEAREST_PRIMARY_CLASS", "Distance of nearest primary class",
  "father_educ", "Father's education"
)

# Summarize selection_df quant vars
# Process data
sel_quant_stats <- selection_df %>%
  summarize(across(
    all_of(sel_quant_meta$var),
    list(
      Min = ~min(.x, na.rm = TRUE),
      `1Q` = ~quantile(.x, .25, na.rm = TRUE),
      Med = ~median(.x, na.rm = TRUE),
      `3Q` = ~quantile(.x, .75, na.rm = TRUE),
      Max = ~max(.x, na.rm = TRUE),
      Mean = ~mean(.x, na.rm = TRUE),
      SD = ~sd(.x, na.rm = TRUE)
    ),
    .names = "{.col}_{.fn}"
  )) %>%
  pivot_longer(
    cols = everything(),
    names_to = c("var", "stat"),
    # Only split off ONE of the 7 stats at the end
  )

```

```

    names_pattern = "^(.*)_ (Min|1Q|Med|3Q|Max|Mean|SD)$"
  ) %>%

  pivot_wider(names_from = stat, values_from = value) %>%
  left_join(sel_quant_meta, by = "var") %>%

# Add number of observation column

  left_join(
    selection_df %>%
    summarize(across(all_of(sel_quant_meta$var),
      ~ sum(!is.na(.x)))) %>%
    pivot_longer(everything(),
      names_to = "var",
      values_to = "N"),
    by="var"
  ) %>%

# Round and format: Set var as factor with intended order, then arrange
  mutate(
    var = factor(var, levels = sel_quant_meta$var),
    across(
      Min:SD,
      ~ case_when(
        var == "ENROLLMENT_COST" ~ comma(.x, accuracy = 0.01),
        TRUE ~ sprintf("%.2f", .x)
      )
    )
  ) %>%
  arrange(var) %>%
  select(var, label, N, Min, `1Q`, Med, `3Q`, Max, Mean, SD)

# Summarize selection_df cat vars

```

```

sel_cat_stats <- selection_df %>%
  select(all_of(sel_cat_meta$var)) %>%
  map_df(~ {
    freq <- table(.x)
    tibble(
      Values = paste(names(freq), collapse=", "),
      Mode = names(freq)[which.max(freq)],
      `% Mode` = round(max(freq)/sum(freq)*100,1),
      `Least Freq.` = names(freq)[which.min(freq)],
      `% Least Freq.` = round(min(freq)/sum(freq)*100,1)
    )
  }, .id="var") %>%
  left_join(sel_cat_meta, by="var") %>%
  # Add number of observations column
  left_join(
    selection_df %>%
    summarize(across(all_of(sel_cat_meta$var),
      ~ sum(!is.na(.x)))) %>%
    pivot_longer(everything(),
      names_to = "var",
      values_to = "N"),
    by="var"
  ) %>%
  mutate(var = factor(var, levels = sel_cat_meta$var)) %>%
  arrange(var) %>%
  select(var, label, N, Values, Mode, `% Mode`, `Least Freq.`, `% Least Freq.`)

```

Chunk 13: Construct 2007–08 measures

```

# EMIE: Percentage of enrolled kids in EMI (code "02"), weighted by child-level weight

```

```

temp <- edu0708b5 %>%
  filter(AGE <= 19) %>%
  mutate(district_code_0708 = as.factor(district_code)) %>%
  group_by(district_code_0708) %>%
  arrange(district_code_0708) %>%
  summarise(
    EMIE = 100*weighted.mean(
      x = (MEDIUM_INSTRUCTION == "02"),
      w = weight,
      na.rm = TRUE
    )
  )

# EMIE = Number of children aged 5-19 enrolled in EMI schools / Total number of children
#   ↪ aged 5-19 enrolled in any school

df0708 <- right_join(districts_0708, temp, by = "district_code_0708")

# Household characteristics from edu0708b3. Weighted by household-level weight
temp <- edu0708b3 %>%
  distinct(district_code, HHID, .keep_all = TRUE) %>%
  mutate(district_code_0708 = as.factor(district_code)) %>%
  group_by(district_code_0708) %>%
  summarise(
    # Total persons in pop.
    npeople_0708 = sum(weight * HH_SIZE, na.rm = TRUE),

    # Total households in pop.
    nhouses_0708 = sum(weight, na.rm = TRUE),

```

```

# Mean per-capita consumption, weighted
consumption_0708 = weighted.mean(
  x = TOTAL / HH_SIZE,
  w = weight,
  na.rm = TRUE
),

# Gini of per-capita consumption, weighted
gini_cons_0708 = Gini(
  x      = TOTAL / HH_SIZE,
  weights = weight
)
) %>%
ungroup()

df0708 <- right_join(df0708, temp, by = "district_code_0708")

# Merge with IMR after it has been aggregated at the district level
temp <- selection_df %>%
  group_by(district_code_0708) %>%
  summarize(
    avg_IMR = weighted.mean(IMR, w = weight, na.rm = TRUE),
    .groups = "drop"
  )

df0708 <- left_join(df0708, temp, by = "district_code_0708")

```

```

# Demographic characteristics, from edu0708b4
# Collapse LAND_POSSESSED_CODE into four bins: no_land, small (0.005-0.20), medium
↳ (0.21-2.00), large (>2.00)
land_breaks <- c("01", c("02","03","04"), c("05","06","07"), c("08","10","11","12"))

temp <- edu0708b4 %>%
  mutate(
    # Recode land categories
    land_cat4 = case_when(
      LAND_POSSESSED_CODE == "01" ~ "no_land",
      LAND_POSSESSED_CODE %in% c("02","03","04") ~ "small",
      LAND_POSSESSED_CODE %in% c("05","06","07") ~ "medium",
      LAND_POSSESSED_CODE %in% c("08","10","11","12") ~ "large",
      TRUE ~ NA_character_
    ),
    # Numeric education
    edu_num = as.integer(na_if(as.character(EDUCATION_LEVEL), "")),
    # Dependent / working-age flags
    is_dep = if_else(AGE <= 14 | AGE >= 65, 1L, 0L),
    is_work = if_else(AGE >= 15 & AGE <= 64, 1L, 0L)
  ) %>%
  group_by(district_code) %>%
  summarise(
    # Pct urban
    pct_urban = 100*sum(weight * (SECTOR == 2)) / sum(weight),

    # Avg household size
    avg_hh_size = sum(weight * HH_SIZE) / sum(weight),

    dependency_ratio = 100*sum(weight * is_dep) / sum(weight * is_work),

```

```

# Female-headed households
pct_fem_head = 100*sum(weight * (SEX == 2 & RELATION_TO_HEAD == 1)) / sum(weight),

# Religion pcts
pct_hindu = 100*sum(weight * (RELIGION == 1)) / sum(weight),
pct_muslim = 100*sum(weight * (RELIGION == 2)) / sum(weight),
pct_other_religion = 100*sum(weight * (RELIGION %in% c(3,4,5,6,7,8))) / sum(weight),

# Caste pcts (ref: others)
pct_st = 100*sum(weight * (SOCIAL_GROUP == "1")) / sum(weight),
pct_sc = 100*sum(weight * (SOCIAL_GROUP == "2")) / sum(weight),
pct_obc = 100*sum(weight * (SOCIAL_GROUP == "3")) / sum(weight),
,

# Land possession pcts (ref: no_land)
pct_small_land = 100*sum(weight * (land_cat4 == "small")) / sum(weight),
pct_medium_land = 100*sum(weight * (land_cat4 == "medium")) / sum(weight),
pct_large_land = 100*sum(weight * (land_cat4 == "large")) / sum(weight),

# Education of head (ref: illiterate)
total_heads = sum(weight * (RELATION_TO_HEAD == 1)),
pct_head_illiterate = 100*sum(weight * (RELATION_TO_HEAD == 1 & edu_num == 1)) /
  ↪ total_heads,
pct_head_lit_to_primary = 100*sum(weight * (RELATION_TO_HEAD == 1 & edu_num >= 2 &
  ↪ edu_num <= 7)) / total_heads,
pct_head_secondary_plus = 100*sum(weight * (RELATION_TO_HEAD == 1 & edu_num >= 8)) /
  ↪ total_heads
) %>%
ungroup() %>%
  rename(district_code_0708 = district_code)

```



```

df0708 <- left_join(df0708, temp, by = "district_code_0708")

temp <- cons0708hhchar %>%
  # If multiple rows per household, keep one
  distinct(HH_ID, .keep_all = TRUE) %>%
  group_by(District) %>%
  # Indicator for if Type_of_structure == "1" (pucca)
  mutate(is_pucca = as.integer(Type_of_structure == "1")) %>%
  # Weighted Percentage
  summarise(
    pct_pucca = 100*sum(Multiplier * is_pucca) / sum(Multiplier),
    .groups = "drop"
  ) %>%
  rename(district_code_0708 = District)

df0708 <- left_join(df0708, temp, by = "district_code_0708")

#### Diagnose issues ####

# edu0708b5 %>% distinct(district_code) %>% nrow() - df0708 %>%
  ↪ distinct(district_code_0708) %>% nrow()
# Number of districts did not change in this operation

missing_districts_0708 <- anti_join(districts_0708, df0708, by = "district_code_0708")
# 7 districts not included.
# Leh (Ladakh), Jammu and Kashmir: Explicitly excluded due to safety concerns

```

```

# Kargil, Jammu and Kashmir: Ditto
# Punch, Jammu and Kashmir: Doesn't have a war named after it like Kargil does, but
  ↳ likely ditto too: instability along Line of Control --> safety concerns, difficult
  ↳ logistics
# New Delhi, Delhi: Representative sampling tough given preponderance of bureaucrats,
  ↳ diplomats, other transient officials
# Central, Delhi: Combo of New Delhi's and Mumbai's reasons
# Mumbai, Maharashtra: Rep. samp. tough given slums, rapid cyclical migration,
  ↳ tightly-secured gated/high-rise residential areas
# Kanniyakumari, Tamil Nadu: Heavy tourist activity (at the southern tip of Indian
  ↳ peninsula) --> difficulties identifying residents, churn in who actually is a
  ↳ resident?
# Will assume sample weights have accounted for the exclusion of these districts

# Dotplot of the ratio (npeople_0708 / nhouses_0708)
plt_nkids_per_house <- ggplot(
  df0708 %>% mutate(
    ratio = npeople_0708 / nhouses_0708,
    diff = npeople_0708 - nhouses_0708
  ),
  aes(x = district_code_0708, y = ratio)) +
  geom_point(size = 3) +
  scale_y_continuous(breaks = pretty_breaks(n = 5)) +
  labs(title = "Average Students Per Household, Per District",
    x = "District (sorted by state and within-state region)", y = "Average Students
  ↳ Per Household") +
  theme_minimal()
# Pretty consistent number of students per household across districts and states!

```

Chunk 14: Construct 2017–18 measures

```
temp <- edu1718b3 %>%  
  # Build district code, pick weights  
  mutate(  
    district_code_1718 = paste0(NSS_Region, District),  
    weight = MULT_Combined  
  ) %>%  
  # One row per household  
  distinct(HHID, .keep_all = TRUE) %>%  
  # Strings into numbers  
  mutate(  
    Household_size = as.numeric(Household_size),  
    HH_Con_exp_rs = as.numeric(HH_Con_exp_rs)  
  ) %>%  
  # Aggregate by district  
  group_by(district_code_1718) %>%  
  summarise(  
    # Number of people  
    npeople_1718 = sum(weight * Household_size, na.rm = TRUE),  
    # Number of households  
    nhouses_1718 = sum(weight, na.rm = TRUE),  
  
    # Mean per-capita consumption  
    consumption_1718 = weighted.mean(  
      x = HH_Con_exp_rs / Household_size,  
      w = weight,  
      na.rm = TRUE  
    ),  
  
    # Weighted Gini of per-capita consumption  
    gini_cons_1718 = Gini(  
      x = HH_Con_exp_rs / Household_size,
```

```

    weights = weight
  )
) %>%
ungroup()

# No households had an NA for consumption

df1718 <- right_join(districts_1718, temp, by = "district_code_1718")

# edu1718b3 %>% distinct(StateDistrict) %>% nrow() - df1718 %>%
  ↪ distinct(district_code_1718) %>% nrow()
# Number of districts did not change in this operation

missing_districts_1718 <- anti_join(districts_1718, df1718, by = "district_code_1718")
# Only Mumbai has no data on it.

```

Chunk 15: Construct 2001 measure (IV)

```

# Preliminary test of IV strength:
# Dotplot of EMIE values by district_code
plt_EMI_across_stats_and_districts <- ggplot(
  df0708 %>% mutate(
    district_prefix = substr(as.character(district_code_0708), 1, 2)
  ),
  aes(x = district_code_0708, y = EMIE, color = district_prefix)
) +
geom_point(size = 3) +
scale_y_continuous(breaks = pretty_breaks(n = 5)) +
labs(title = "Percentage of Students in EMI by District, 2007-08",
      x = "District (sorted by state and within-state region)", y = "Percentage in EMI",

```

```

    color = "State Code") +
  theme_minimal()

# EMIE has three peaks: in Jammu and Kashmir; in Sikkim, Arunachal Pradesh, Nagaland,
  ↳ Manipur, Mizoram, Tripura, maybe Meghalaya; and in Andhra Pradesh, Karnataka, Goa,
  ↳ Lakshadweep, Kerala, Tamil Nadu, Pondicheri, and Andaman & Nicobar! The regions which
  ↳ historically were the furthest from Hindi!

# Many districts in the second group seem to have EMIE around 1, and the range of EMIE
  ↳ outside peaks is between 0.4 and 0.1. Justification for looking at smaller units of
  ↳ analysis?


# Process the dataframe and compute the weighted average ling_distance for each (state,
  ↳ district) group.

mother_tongues_01 <- census01 %>%

  # Group by state and district to ensure that same district names in different states
  ↳ are treated separately

  group_by(state, district) %>%

  # For each group, keep only the top three rows by spkr_tot

  slice_max(order_by = spkr_tot, n = 3) %>%

  # Ungroup before applying new transformations

  ungroup() %>%

  # Create the ling_distance column based on the mother_tongue values and @shastry2012a's
  ↳ 0-5 measure of degrees of linguistic distance

  mutate(
    ling_degrees = case_when(
      mother_tongue %in% c("Hindi", "Urdu") ~ 0,
      mother_tongue %in% c("Gujarati", "Punjabi", "Rajasthani") ~ 1,
      mother_tongue %in% c("Konkani", "Marathi") ~ 2,
      mother_tongue %in% c("Assamese", "Bengali", "Bihari", "Oriya") ~ 3, # Oriya was
      ↳ renamed as Odia in 2011

      mother_tongue %in% c("Kashmiri", "Sindhi", "Sinhalese") ~ 4,

```

```

    TRUE ~ 5 # Why TRUE?
  )
) %>%

# Group again by state and district to compute group summaries
group_by(state, district) %>%

# Calculate the weighted average ling_distance for each group. For each language  $\ell$ 
  ↪ which is among the top three most spoken in district  $d$ :
  ↪ 
$$\frac{\sum_{\ell} (\text{linguistic distance of } \ell \times \text{num. speakers of } \ell \text{ in } d)}{\sum_{\ell} (\text{num. speakers of } \ell)}$$

  summarize(
    wavg_ling_degrees = sum(ling_degrees * spkr_tot, na.rm = TRUE) / sum(spkr_tot, na.rm
      ↪ = TRUE),
    .groups = "drop" # Makes ungroup() redundant?
  ) %>%
ungroup() %>%
rename(state_01 = state, district_01 = district)

# census01 %>% mutate(StateDistrict = paste0(state_code, district_code)) %>%
  ↪ distinct(StateDistrict) %>% nrow() - mother_tongues_01 %>% mutate(StateDistrict =
  ↪ paste0(state_01, district_01)) %>% distinct(StateDistrict) %>% nrow()
# Number of districts did not change in this operation

```

Chunk 16: Match districts: Test joining methods

```

# Number of rows from fuzzy full joining district_tracker (734 rows) and
  ↪ mother_tongues_01 (593 rows) with different metrics:
# "osa": 859
# "lv": 859
# "dl": 859
# "hamming": 872
# "lcs": 825

```

```

# "qgram": 829
# "cosine": 435,262
# "jaccard": 435,262
# "jw": 435,262
# See https://cran.r-project.org/web/packages/stringdist/stringdist.pdf#page=23 for more

# Testing different methods of fuzzy joining

# # With ("lcs", "osa"), (3, 3):
# colSums(is.na(joined_df))
# # 75 NAs in 01 columns, 124 in 07-08, 105 in 17-18
# sum(apply(joined_df, 1, anyNA))
# # 189/734 rows have an NA
#
# # With ("jw", "dl", "osa", "lcs"), (0.10, 2, 3, 5)
# colSums(is.na(joined_df))
# # 70 in 01, 119 in 07-08, 98 in 17-18
# sum(apply(joined_df, 1, anyNA))
# # 180/734
#
# methods <- c("soundex", "jw", "dl", "osa", "lcs")
# thresholds <- c(0, 0.15, 2, 1, 5)
# colSums(is.na(joined_df))
# # 47 in 01, 105 in 07-08, 83 in 17-18
# sum(apply(joined_df, 1, anyNA))
# # 158/734
# # With lcs, the higher the better. Would need normalized version like 1 -
# ↪ lcs(a,b)/min(nchar(a),nchar(b))

```

```

# # More manual tests
# stringdist("East Godavari", "Godavari East", method = "qgram")
# # 0
# stringdist("East Godavari", "Godavari East", method = "jaccard")
# # 0

evaluate_distances <- function(pairs,
                               methods, thresholds,
                               col1 = "str1", col2 = "str2") {
  if(length(methods) != length(thresholds)) {
    stop("\"methods\" and \"thresholds\" must have the same length.")
  }
  # Ensure chr type
  pairs <- pairs %>%
    mutate(
      !!col1 := as.character(.data[[col1]]),
      !!col2 := as.character(.data[[col2]])
    )

  # Compute distances for each method
  out <- bind_rows(
    lapply(seq_along(methods), function(i) {
      m <- methods[i]
      th <- thresholds[i]

      pairs %>%
        transmute(
          str1 = .data[[col1]],
          str2 = .data[[col2]],
          method = m,
          distance = stringdist(str1, str2, method = m),

```



```

        threshold = th,
        match = distance <= th
    )
  })
)

# Keep a consistent ordering
out %>% arrange(str1, str2, method)
}

# Some sample pairs
pairs <- tribble(
  ~str1, ~str2,
  "Baleshwar", "Balasore",
  "Jammu & Kashmir", "Jammu and Kashmir",
  "East Godavari", "Godavari East",
  "Sikim", "Sikkim",
  "Mumbai", "Mumbai",
  "24-Parganas ( North )", "North Twenty Four Parganas",
  "North Twenty Four Pargan*", "North Twenty Four Parganas",
  "Sahibzada Ajit Singh Nag*", "Sahibzada Ajit Singh Nagar",
  "Sri Potti Sriramulu Nell*", "Sri Potti Sriramulu Nellore"
)

#### Helper vectors ####
methods <- c("soundex", "qgram", "jw", "dl", "osa")
thresholds <- c(0, 0, 0.15, 2, 1)

# soundex=0 --> Phonetic variants in anglicization allowed (if coarsely measured)
# qgram = 0 --> Some rearrangements of characters allowed
# jw<=0.15 --> Respellings + vowel swaps with 0.85 similarity allowed
# dl<=2 --> <=2 insertions + deletions + substitutions + transpositions allowed

```

```

# osa<=1 --> 1 typo allowed
# Template for testing these vs. other methods/thresholds can be found at start of chunk

# Evaluate distances and test joining methods
# colSums(is.na(joined_df))
# # 53 in 01, 112 in 07-08, 89 in 17-18
# sum(apply(joined_df, 1, anyNA))
# # 166/734
# evaluate_distances(pairs, methods, thresholds)

```

Chunk 17: Match districts: Define joining functions

```

# A. Loop over the methods*thresholds
# B. For each methods*thresholds pair, fuzzy full join all still-unmatched rows
# C. Return joined, unmatched_df1, unmatched_df2
fuzzy_join_sequence <- function(df1, df2,
                                dist1, state1,
                                dist2, state2,
                                methods, thresholds, mode = "full") {
  # Attach row IDs to track unmatched
  df1_id <- df1 %>% mutate(.id1 = row_number())
  df2_id <- df2 %>% mutate(.id2 = row_number())

  df1_curr <- df1_id
  df2_curr <- df2_id
  matched_all <- tibble()

  for(i in seq_along(methods)) {
    full_j <- stringdist_join(
      df1_curr, df2_curr,

```

```

    by = setNames(c(dist2, state2), c(dist1, state1)),
    mode = mode,
    method = methods[i],
    max_dist = thresholds[i],
    distance_col = "dist"
  )

  # Keep only pairs (i.e., those with both sides present) and rank them
  matched_i <- full_j %>%
    filter(!is.na(.id1) & !is.na(.id2)) %>%
    group_by(.id1) %>%
    slice_min(order_by = dist, n = 1, with_ties = FALSE) %>%
    ungroup() %>%
    group_by(.id2) %>%
    slice_min(order_by = dist, n = 1, with_ties = FALSE) %>%
    ungroup()
  matched_all <- bind_rows(matched_all, matched_i)

  # Remove them from the to-do sets
  df1_curr <- df1_curr %>% filter(!(.id1 %in% matched_i$.id1))
  df2_curr <- df2_curr %>% filter(!(.id2 %in% matched_i$.id2))
}

list(
  joined = matched_all,
  unmatched_df1 = df1_curr %>% select(-.id1),
  unmatched_df2 = df2_curr %>% select(-.id2)
)
}

# Default is full join so that this can be used for both matching and diagnostics

```

```

# For each df name in input vector:
# A. Extract its "01"/"05"/... suffix
# B. Find the suffix's district_/state_ columns in joined_df.
# C. Run the fuzzy join sequence on each column. (Recall justification of fuzzy joining
↳ states in Chunk 6: Match districts: Construct district tracking dfs.)
# D. Repeat with the predecessor suffix.
merge_dfs_into_tracker <- function(
  dfs,
  tracker,
  years_of_interest,
  methods, thresholds,
  flag = FALSE
) {
  # 1. Initialize both joined_df and (if flag=TRUE) flagged_df
  joined_df <- tracker
  if (flag) {
    flagged_df <- tracker
    # Prepare placeholder vector for each df-name
    matched_lists <- lapply(names(dfs), function(x) rep(NA_character_, nrow(tracker)))
    names(matched_lists) <- names(dfs)
  }

  year_suffixes <- substr(as.character(years_of_interest), 3, 4)
  unmatched_list <- vector("list", length(dfs))
  names(unmatched_list) <- names(dfs)

  # 2. Loop over each auxiliary data-frame name
  for (d_name in names(dfs)) {
    df_d <- dfs[[d_name]]
    suffix <- stringr::str_extract(d_name, "\\d{2}$")
    stopifnot(!is.na(suffix))
  }
}

```

```

# Find exactly one district_XX / state_XX in df_d
dist_d <- grep("^district_", names(df_d), value=TRUE) %>%
  grep("code", ., invert=TRUE, value=TRUE)
state_d <- grep("^state_", names(df_d), value=TRUE) %>%
  grep("code", ., invert=TRUE, value=TRUE)
stopifnot(length(dist_d)==1, length(state_d)==1)

df_unmatched <- df_d
if (flag) mv <- matched_lists[[d_name]]

# suffix_chain sorted by absolute distance from said suffix
year_nums <- as.integer(year_suffixes)
suffix_num <- as.integer(suffix)
order_idx <- order(abs(year_nums - suffix_num))
suffix_chain <- year_suffixes[order_idx]

# Work through suffixes, closest to furthest
for (suf in suffix_chain) {
  dist_cols_j <- grep(paste0("^district_", suf, "(?:$|_)"),
    names(joined_df), value=TRUE)
  state_cols_j <- grep(paste0("^state_", suf, "(?:$|_)"),
    names(joined_df), value=TRUE)

  for (dist_col_j in dist_cols_j) {
    ending <- sub(paste0("^district_", suf), "", dist_col_j)
    state_col_j <- paste0("state_", suf, ending)
    if (!state_col_j %in% state_cols_j) next
  }

  # 2a. Multi-threshold fuzzy join
  res <- fuzzy_join_sequence(
    df_unmatched, joined_df,
    dist1 = dist_d,

```

```

state1 = state_d,
dist2 = dist_col_j,
state2 = state_col_j,
methods = methods,
thresholds = thresholds,
mode = "full"
)

# 2b. Pull other cols into joined_df
other_cols <- setdiff(names(df_d), c(dist_d, state_d))
if (nrow(res$joined)>0 && length(other_cols)>0) {
  extra <- res$joined %>%
    select(.id2, all_of(intersect(other_cols, names(res$joined)))) %>%
    arrange(dist, .id2) %>%
    group_by(.id2) %>%
    slice_min(order_by = dist, n = 1, with_ties = FALSE) %>%
    ungroup()

  # Merge into joined_df
  joined_df <- joined_df %>%
    mutate(.row = row_number()) %>%
    left_join(extra, by = c(".row"=".id2")) %>%
    select(-.row)

  # If flag, merge into flagged_df
  if (flag) {
    flagged_df <- flagged_df %>%
      mutate(.row = row_number()) %>%
      left_join(extra, by = c(".row"=".id2")) %>%
      select(-.row)
  }
}

```

```

# 2c. If flag=TRUE, record which district got matched this iteration
if (flag && nrow(res$joined)>0) {
  col_match <- grep(
    paste0("^", dist_d, "(?:$|\\.x$)"),
    names(res$joined), value=TRUE
  )[1]
  info <- res$joined[, c(".id2", col_match), drop=FALSE]
  colnames(info)[2] <- "match_dist"
  info <- info %>%
    group_by(.id2) %>%
    slice_head(n = 1) %>%
    ungroup()

  idx <- info$.id2
  new_vals <- info$match_dist
  blank <- is.na(mv[idx])
  mv[idx[blank]] <- new_vals[blank]
}

# 2d. Update the df of unmatched
df_unmatched <- res$unmatched_df1
if (nrow(df_unmatched)==0) break
}

if (nrow(df_unmatched)==0) break
}

# 2e. Once done, append flag-column to flagged_df
if (flag) {
  flagged_df[[d_name]] <- mv
}
unmatched_list[[d_name]] <- df_unmatched

```

```

    if (nrow(df_unmatched)>0) {
      warning(sprintf("Some rows in %s were never matched", d_name))
    }
  }

# 3. Finalize flagged_df if flag=TRUE
if (flag) {
  max_suf <- year_suffixes[which.max(as.integer(year_suffixes))]
  keep_cols <- names(flagged_df)[
    (grepl("^district_", names(flagged_df)) & !grepl("code", names(flagged_df))) |
    names(flagged_df) %in% names(dfs) |
    names(flagged_df) == paste0("state_", max_suf)
  ]
  flagged_df <- flagged_df[, keep_cols, drop=FALSE]
}

# 4. Return!!!
if (flag) {
  list(
    joined_df = joined_df,
    unmatched = unmatched_list,
    flagged_df = flagged_df
  )
} else {
  list(
    joined_df = joined_df,
    unmatched = unmatched_list
  )
}
}

```


Chunk 18: Match districts: Manually fix errors

```
# Errors were identified in Chunk 20: Match districts: Diagnose errors
```

```
mother_tongues_01 <- mother_tongues_01 %>%  
  mutate(  
    district_01 = recode(district_01,  
      "Sant Ravidas Nagar Bhadohi" = "Bhadohi",  
      "Kanker" = "Uttar Bastar Kanker"  
    ),  
    state_01 = recode(state_01,  
      "N.c.t. Of Delhi" = "Delhi"  
    )  
  )  
  
df0708 <- df0708 %>%  
  mutate(  
    district_0708 = recode(district_0708,  
      "Sahib Mansa" = "Sahibzada Ajit Singh Nagar",  
      "J Phule Nagar" = "Jyotiba Phule Nagar",  
      "G. Buddha Nagar" = "Gautam Buddha Nagar",  
      "S. Kabir Nagar" = "Sant Kabir Nagar",  
      "S R Nagar (Bhadohi)" = "Bhadohi",  
      "Champaran (W)" = "Pashchim Champaran",  
      "Champaran (E)" = "Purba Champaran",  
      "North (Mongam)" = "North",  
      "West (Gyalshing)" = "West",  
      "South (Nimachai)" = "South",  
      "East (Gangtok)" = "East",  
      "West Dinajpur" = "Dakshin Dinajpur",  
      "24-Parganas ( North )" = "North Twenty Four Parganas",  
      "24-Parganas ( South )" = "South Twenty Four Parganas",
```

```

    "Singhbhum(W)" = "Pashchimi Singhbhum",
    "Singhbhum(E)" = "Purbi Singhbhum",
    "W. Nimar ( Khargoan )" = "West Nimar",
    "E. Nimar ( Khandwa )" = "East Nimar",
    "Kanker" = "Uttar Bastar Kanker"
  )
)

df1718 <- df1718 %>%
  mutate(
    district_1718 = recode(district_1718,
      "Sant Ravidas Nagar(Bhadohi)" = "Bhadohi",
      "North District" = "North",
      "West District" = "West",
      "South District" = "South",
      "East District" = "East",
      "Khargone (West Nimar)" = "West Nimar",
      "Khandwa (East Nimar)" = "East Nimar",
      "Leh" = "Leh (Ladakh)",
      "Y.S.R. (Cuddapah)" = "Cuddapah",
      "Rajanna" = "Rajanna Sircilla"
    ),
    state_1718 = recode(state_1718,
      "A & N Islands" = "Andaman and Nicobar Islands"
    )
  )

districts_20 <- districts_20 %>%
  mutate(
    district_20 = recode(district_20,

```

```

    "North District" = "North", # Note the two spaces!!
    "West District" = "West",
    "South District" = "South",
    "East District" = "East",
    "Leh" = "Leh (Ladakh)",
    "Cooch Behar" = "Koch Bihar",
    "Y.S.R." = "Y.S.R. Kadapa"
  ),
  state_20 = recode(state_20,
    "Ladakh" = "Jammu and Kashmir"
  )
)

```

```

district_timeseries <- district_timeseries %>%
  mutate(
    district_24 = recode(district_24,
      "Shamli (Prabuddhanagar)" = "Shamli",
      "Y.S.R." = "Y.S.R. Kadapa"
    ),
    district_01 = recode(district_01,
      "Kanker" = "Uttar Bastar Kanker"
    )
  )

```

```

district_tracker <- district_tracker %>%
  mutate(

    district_01 = recode(district_01,
      "West Champaran" = "Pashchim Champaran",
      "East Champaran" = "Purba Champaran",

```

```

"North District" = "North",
"West District" = "West",
"South District" = "South",
"East District" = "East",
"North 24 Parganas" = "North Twenty Four Parganas",
"South 24 Parganas" = "South Twenty Four Parganas",
"West Singhbhum" = "Pashchimi Singhbhum",
"East Singhbhum" = "Purbi Singhbhum",
"Leh" = "Leh (Ladakh)",
"Cooch Behar" = "Koch Bihar",
"Dima Hasao" = "North Cachar Hills",
"Kaimur" = "Kaimur (Bhabua)",
"Kabeerdham" = "Kawardha",
"Kutch" = "Kachchh",
"Dang" = "The Dangs",
"Belagavi" = "Belgaum",
"Shivamogga" = "Shimoga",
"Khandwa" = "East Nimar",
"Khargone" = "West Nimar",
"Subarnapur" = "Sonapur",
"Amroha" = "Jyotiba Phule Nagar",
"Lakhimpur Kheri" = "Kheri",
"Central Delhi" = "Central",
"East Delhi" = "East",
"North Delhi" = "North",
"North East Delhi" = "North East",
"North West Delhi" = "North West",
"South Delhi" = "South",
"South West Delhi" = "South West",
"West Delhi" = "West"
),

```

```

district_06 = recode(district_06,
  "S.A.S. Nagar" = "Sahibzada Ajit Singh Nagar"
),

district_08 = recode(district_08,
  "S.P.S. Nellore" = "Sri Potti Sriramulu Nellore",
  "Aizawl" = "Saitual",
  "Pauri Garhwal" = "Garhwal"
)
)

# Alternative future strat:

# Build a tibble of corrections
# corrections <- tribble(~old, ~new, "Sant Ravidas Nagar Bhadohi", "Bhadohi", "Kanker",
  ↪ "Uttar Bastar Kanker") # etc.

# mother_tongues_01 <- mother_tongues_01 %>%
#   left_join(corrections, by = c("district_01" = "old")) %>%
#   mutate(
#     district_01 = coalesce(new, district_01)
#   ) %>%
#   select(-new)

```

Chunk 19: Match districts: Run the joining functions

```

# Helper vectors:
dfs <- list(mother_tongues_01, df0708, df1718, districts_20)

# Also years_of_interest, year_suffixes, methods, and thresholds

```

```

# Join data with district_timeseries data
temp <- merge_dfs_into_tracker(
  dfs = dfs, tracker = district_timeseries,
  years_of_interest = c("2001", "2011", "2024"),
  methods = methods, thresholds = thresholds,
  flag = TRUE
)

joined_df_timeseries <- temp$joined_df

unmatched_df_timeseries <- bind_rows(temp$unmatched, .id = "source")

flagged_df_timeseries <- temp$flagged_df


# Join data with district_tracker
temp <- merge_dfs_into_tracker(
  dfs = dfs, tracker = district_tracker,
  years_of_interest = years_of_interest,
  methods = methods, thresholds = thresholds,
  flag = TRUE
)

joined_df_tracker <- temp$joined_df

unmatched_df_tracker <- bind_rows(temp$unmatched, .id = "source")

flagged_df_tracker <- temp$flagged_df

```

Chunk 20: Match districts: Diagnose errors

```
# Errors are fixed in Chunk 18: Match districts: Manually fix errors

# Change dfs for diagnostic purposes, if desired
dfs <- list(mother_tongues_01, df0708, df1718, districts_20)

# Function to extract mismatches:
unmatched_rows <- function(
  dfs, tracker,
  years_of_interest,
  methods, thresholds
) {
  # Compute two-digit suffixes
  suffixes <- substr(as.character(years_of_interest), 3, 4)

  # Run merge_dfs_into_tracker on each df, extract unmatched_df, add source
  result_list <- lapply(names(dfs), function(nm) {
    out <- merge_dfs_into_tracker(
      dfs = dfs[nm], tracker = tracker,
      years_of_interest = years_of_interest,
      methods = methods, thresholds = thresholds
    )$unmatched[[nm]]

    # Add source column of length nrow(out)
    out <- out %>% mutate(source = nm)
    out
  })

  # Bind their rows
  combined <- bind_rows(result_list)

  # Coalesce the district/state_ columns
```

```

state_cols <- combined %>%
  select(starts_with("state_"), -contains("code")) %>%
  names()
dist_cols <- combined %>%
  select(starts_with("district_"), -contains("code")) %>%
  names()

combined %>%
  mutate(
    district = do.call(coalesce, select(., all_of(dist_cols))),
    state = do.call(coalesce, select(., all_of(state_cols)))
  ) %>%
  select(district, state, source)
}

```

Compare outcomes with different tracker dfs

```

# Identify errors for each tracker df
unmatched_rows_timeseries <- unmatched_rows(
  dfs = dfs, tracker = district_timeseries,
  years_of_interest = c("2001", "2011", "2024"),
  methods = methods, thresholds = thresholds
)

unmatched_rows_tracker <- unmatched_rows(
  dfs = dfs, tracker = district_tracker,
  years_of_interest = c("2001", "2011", "2024"),
  methods = methods, thresholds = thresholds
)

```



```

# Combine all errors
unmatched_rows <- full_join(unmatched_rows_timeseries, unmatched_rows_tracker, by =
  ↪ join_by(state, district), suffix = c("_timeseries", "_tracker"))

# Create a df with all rows from original data, so I can search for close matches easily

all_rows <- bind_rows(
  lapply(names(dfs), function(nm) dfs[[nm]] %>% mutate(source = nm)),
  missing_districts_1718 %>% mutate(source = "missing_districts_1718"),
  missing_districts_0708 %>% mutate(source = "missing_districts_0708"),
  district_timeseries %>% mutate(source = "district_timeseries"),
  district_tracker %>% mutate(source = "district_tracker")
) %>%
mutate(
  state = coalesce(
    !!!select(., starts_with("state_"), -contains("code"))
  ),
  district = coalesce(
    !!!select(., starts_with("district_"), -contains("code"))
  )
) %>%
select(state, district, source)

#### Correct even more NAs ####

#flagged_df_timeseries %>% .[!complete.cases(.),] %>% nrow()
# 155 rows

#joined_df_timeseries %>% .[!complete.cases(.),] %>% nrow()

```

```

# 305 rows

#flagged_df_tracker %>% .[!complete.cases(.),] %>% nrow()
# 92 rows

#joined_df_tracker %>% .[!complete.cases(.),] %>% nrow()
# 250 rows

# ***Why the gigantic discrepancy???

```

Chunk 21: Match districts: Join with geometry data

```

# Join geometry

# For merged data based on district_tracker
joined_df_tracker <- district_bnds_20 %>%
  select(JID, geometry) %>%
  full_join(
    joined_df_tracker,
    by = "JID"
  )

joined_df_tracker <- joined_df_tracker %>%
  mutate(
    consumption_pct_change = (consumption_1718 - consumption_0708) / consumption_0708 *
      ↪ 100,
    gini_change = gini_cons_1718 - gini_cons_0708
  )

joined_df_tracker <- joined_df_tracker %>%

```

```

left_join(region_df, by = "state_20")

# For merged data based on district_timeseries
joined_df_timeseries <- district_bnds_20 %>%
  select(JID, geometry) %>%
  full_join(
    joined_df_timeseries,
    by = "JID"
  )

joined_df_timeseries <- joined_df_timeseries %>%
  mutate(
    consumption_pct_change = (consumption_1718 - consumption_0708) / consumption_0708 *
      ↪ 100,
    gini_change = gini_cons_1718 - gini_cons_0708
  )

joined_df_timeseries <- joined_df_timeseries %>%
  left_join(region_df, by = join_by(state_24 == state_20))

#### Compare districts in district_tracker merge vs. district_timeseries merge ####

# tm_shape(joined_df_timeseries) + tm_fill("consumption_pct_change", palette = "Reds",
  ↪ title = "%ΔConsumption") + tm_borders(alpha = 0.2) + tm_layout(frame = FALSE)

# tm_shape(joined_df_tracker) + tm_fill("consumption_pct_change", palette = "Reds", title
  ↪ = "%ΔConsumption") + tm_borders(alpha = 0.2) + tm_layout(frame = FALSE)

```

Chunk 22: Geospatial: Make maps

```
# For the names of all color palette:
#cols4all::c4a_palettes()

# To explore the palettes:
#cols4all::c4a_gui()

# To plot a palette:
#cols4all::c4a("zissou1continuous", 7) %>% cols4all::c4a_plot()

# To view the color codes of a palette:
#cols4all::c4a("rd_yl_bu", 5)

# To explore some of the palettes supported by tmap:
# tmaptools::palette_explorer()

joined_df <- joined_df_tracker

# Based on geoemetry from @bhatiaMergingUpdatedDistrictlevel2020, and code from
↳ @nowosadElegantInformativeMaps2021

asp = 0

map_consumption_pct_change <- tm_shape(joined_df) +
  tm_fill("consumption_pct_change", palette = "brewer.reds", title = "%Δ Consumption",
    ↳ style = "cont", legend.hist = FALSE) +
  tm_borders(alpha = 0.2) +
  tm_layout(frame = FALSE, asp = asp)
# tmap_save(tm = map_consumption_pct_change, filename = "map_consumption_pct_change.png",
  ↳ width = 8, height = 6, units = "in", dpi = 300) # dots per inch (300 dpi is print
  ↳ quality)
# Recall that this change does not account for inflation!

map_EMIE <- tm_shape(joined_df) +
  tm_fill("EMIE", palette = "brewer.blues", title = "EMI Exposure", style = "fixed",
    ↳ breaks = c(0, 2.5, 10, 25, 50, 100), legend.hist = TRUE) +
```

```

tm_borders(alpha = 0.2) +
tm_layout(frame = FALSE, asp = asp)
# tmap_save(tm = map_EMIE, filename = "map_EMIE.png", width = 8, height = 6, units =
  ↪ "in", dpi = 300) # dots per inch (300 dpi is print quality)
# To help determine where breaks should be:
# tapply((joined_df %>% drop_na(EMIE))$EMIE, (kmeans((joined_df %>% drop_na(EMIE))$EMIE,
  ↪ centers = 6))$cluster, range)

map_pucca <- tm_shape(joined_df) +
  tm_fill("pct_pucca", palette = "brown", title = "% Pucca Homes", legend.hist = TRUE) +
  tm_borders(alpha = 0.2) +
  tm_layout(frame = FALSE, asp = asp)
# tmap_save(tm = map_pucca, filename = "map_pucca.png", width = 8, height = 6, units =
  ↪ "in", dpi = 300) # dots per inch (300 dpi is print quality)

map_pct_head_secondary_plus <- tm_shape(joined_df) +
  tm_fill("pct_head_secondary_plus", palette = "brewer.greens", title = "% HH Head w/
  ↪ Sec.+", style = "cont", legend.hist = FALSE) +
  tm_borders(alpha = 0.2) +
  tm_layout(frame = FALSE, asp = asp)
# tmap_save(tm = map_pct_head_secondary_plus, filename =
  ↪ "map_pct_head_secondary_plus.png", width = 8, height = 6, units = "in", dpi = 300) #
  ↪ dots per inch (300 dpi is print quality)

map_region <- tm_shape(joined_df) +
  tm_fill("region", palette = "brewer.dark2", title = "Region", legend.hist = TRUE) +
  tm_borders(alpha = 0.2) +
  tm_layout(frame = FALSE, asp = asp)

map_ling_dist <- tm_shape(joined_df) +
  tm_fill("wavg_ling_degrees", palette = "carto.emrld", title = "Linguistic Distance",
  ↪ legend.hist = TRUE) +

```

```

tm_borders(alpha = 0.2) +
tm_layout(frame = FALSE, asp = asp)

# To print all of the maps:
# for(nm in ls(pattern = "^map_")) {print(get(nm))}

#### Save the maps ####
# Uncomment when/if final_draft == TRUE

# width = 6
# height = 5
# # Save each tmap object as a PNG file
# tmap_save(map_EMIE, "map_EMIE.png", width = width, height = height, units = "in", dpi =
  ↪ 300)
# tmap_save(map_consumption_pct_change, "map_consumption.png", width = width, height =
  ↪ height, units = "in", dpi = 300)
# tmap_save(map_pucca, "map_pucca.png", width = width, height = height, units = "in", dpi
  ↪ = 300)
# tmap_save(map_pct_head_secondary_plus, "map_edu.png", width = width, height = height,
  ↪ units = "in", dpi = 300)
# tmap_save(map_region, "map_region.png", width = width, height = height, units = "in",
  ↪ dpi = 300)
# tmap_save(map_ling_dist, "map_ling_dist.png", width = width, height = height, units =
  ↪ "in", dpi = 300)

```

Chunk 23: Geospatial: Make map collages

```

# Read the images
img1 <- image_read("map_EMIE.png")

```

```

img2 <- image_read("map_consumption.png")
img3 <- image_read("map_pucca.png")
img4 <- image_read("map_edu.png")

# Set a standard width, then pad all to a common size
target_width <- 1200
target_height <- 900

# Resize each image by width while preserving aspect ratio
img1 <- image_resize(img1, geometry = paste0(target_width))
img2 <- image_resize(img2, geometry = paste0(target_width))
img3 <- image_resize(img3, geometry = paste0(target_width))
img4 <- image_resize(img4, geometry = paste0(target_width))

# # Then pad with white (or transparent) space to ensure equal height & width for
# alignment
# img1 <- image_extent(img1, paste0(target_width, "x", target_height), gravity =
#   "center", color = "white")
# img2 <- image_extent(img2, paste0(target_width, "x", target_height), gravity =
#   "center", color = "white")
# img3 <- image_extent(img3, paste0(target_width, "x", target_height), gravity =
#   "center", color = "white")
# img4 <- image_extent(img4, paste0(target_width, "x", target_height), gravity =
#   "center", color = "white")

# # Combine
top_row <- image_append(c(img1, img2))
bottom_row <- image_append(c(img3, img4))
collage <- image_append(c(top_row, bottom_row), stack = TRUE)
#collage <- image_append(c(img1, img2, img3, img4), stack = TRUE)

image_write(collage, path = "collage1_map.png")

```

```

# Read the images
img5 <- image_read("map_region.png")
img6 <- image_read("map_ling_dist.png")

# Resize each image by width while preserving aspect ratio
img5 <- image_resize(img5, geometry = paste0(target_width))
img6 <- image_resize(img6, geometry = paste0(target_width))

# # Then pad with white (or transparent) space to ensure equal height & width for
  ↪ alignment
# img5 <- image_extent(img5, paste0(target_width, "x", target_height), gravity =
  ↪ "center", color = "white")
# img6 <- image_extent(img6, paste0(target_width, "x", target_height), gravity =
  ↪ "center", color = "white")

# Combine
collage <- image_append(c(img5, img6), stack = FALSE)

image_write(collage, path = "collage2_map.png")

```

Chunk 24: Geospatial: Neighbor list construction

```

joined_df <- joined_df %>% drop_na()

# # Test results under different contiguity measures; code based on
  ↪ @liChapter14Spatial2019
# remotes::install_github("spatialanalysis/sfExtras")
# library(sfExtras)

```



```

# rook_neighbors <- joined_geom %>% st_rook()
# rook_neighbors %>% lengths() %>% mean()
# # 4.780165 average neighbors per district
# queen_neighbors <- joined_geom %>% st_queen()
# queen_neighbors %>% lengths() %>% mean()
# # 4.783471
# # Average neighbor counts are nearly equivalent

# # Test for fastest contiguity measure
# start.time <- Sys.time()
nb_2020 <- joined_df %>%
  poly2nb(queen = FALSE)
# end.time <- Sys.time()
# end.time - start.time
# # With queen = TRUE: 32.64976 secs
# # With queen = FALSE: 31.15051 secs
# # Rook contiguity slightly faster

# Binary adjacency matrix. Rook-contiguity neighbor list used for speed, conceptual
#  → interpretability
W_2020 <- nb2mat(nb_2020, style = "B", zero.policy = TRUE)
# W_2020[i,j] == 1 if districts i and j share a border, 0 else

```

Chunk 25: 2SLS: Controls and 2SLS formula

```

#### Control vectors ####
controls_needlag <- c("consumption_0708", "gini_cons_0708")
controls_nolag <- c(
  "pct_urban", "avg_hh_size", "dependency_ratio",
  "pct_fem_head",
  "pct_hindu", "pct_muslim",

```

```

    "pct_st", "pct_sc", "pct_obc",
    "pct_small_land", "pct_medium_land", "pct_large_land",
    "pct_head_lit_to_primary",
    "pct_head_secondary_plus")

# Extract spatial lags
controls_lagged <- paste0("W_", controls_needlag)

#### IV Formula ####
make_iv_formula <- function(dep, endog, exog, inst) {
  as.formula(paste0(
    dep, " ~ ",
    paste(c(endog, exog), collapse = " + "),
    " | ",
    paste(c(inst, exog), collapse = " + ")
  ))
}

```

Chunk 26: 2SLS: Baseline 2SLS models

```

# Formula syntax: outcome ~ endogenous + controls | instrument + controls
model_consumption_iv <- ivreg(
  make_iv_formula(
    dep = "consumption_pct_change",
    endog = "EMIE",
    exog = c(controls_needlag, controls_nolag),
    inst = "wavg_ling_degrees"
  ),
  data = joined_df
)

```

```

# First-stage regression
first_stage_consumption <- lm(
  reformulate(
    c("wavg_ling_degrees", controls_needlag, controls_nolag),
    response = "EMIE"
  ),
  data = joined_df # Complete cases already enforced, so ivreg() and lm() will use the
  ↪ same estimation sample
)

# Cluster SEs by state
vcov_first_stage_consumption <- vcovCL(first_stage_consumption, cluster = ~ state_20)
vcov_model_consumption_iv <- vcovCL(model_consumption_iv, cluster = ~ state_20)

# Repeat for Gini model

model_gini_iv <- ivreg(
  make_iv_formula(
    dep = "gini_change",
    endog = "EMIE",
    exog = c(controls_needlag, controls_nolag),
    inst = "wavg_ling_degrees"
  ),
  data = joined_df
)

vcov_model_gini_iv <- vcovCL(model_gini_iv, cluster = ~ state_20)

```

Chunk 27: 2SLS: Make summary tables

```
## joined_df meta:
joined_meta <- tribble(
  ~var, ~label, ~desc,
  "EMIE", "EMIE", "EMI exposure",

  # 2007-08
  "wavg_ling_degrees", "Ling. Distance", "Average linguistic distance of mother tongue
  ↪ from Hindi",
  "npeople_0708", "Population", "Estimated via NSS sample weights",
  "consumption_0708", "Consumption", "Average household monthly consumption expenditures
  ↪ (Rs.)",
  "gini_cons_0708", "Gini of Consumption", "Gini coefficient of consumption",
  "pct_urban", "Pct. Urban", "Percentage of people in an urban area",
  "avg_hh_size", "Avg. HH Size", "Average household size",
  "dependency_ratio", "Dependency Ratio × 100", "Ratio of dependents (0-14, 65+) to labor
  ↪ force (15-64), × 100",
  "pct_fem_head", "Pct. Female Head", "Percentage of households with a female head",
  "pct_hindu", "Pct. Hindu", "Percentage of Hindus",
  "pct_muslim", "Pct. Muslim", "Percentage of Muslims",
  "pct_other_religion", "Pct. Other", "Percentage not Hindu/Muslim",
  "pct_st", "Pct. ST", "Scheduled Tribe",
  "pct_sc", "Pct. SC", "Scheduled Caste",
  "pct_obc", "Pct. OBC", "Other Backward Class",
  "pct_small_land", "Pct. Small Land-Owner", "Owns 0.005-0.40 hectares",
  "pct_medium_land", "Pct. Med. Land-Owner", "Owns 0.41-3.00 hectares",
  "pct_large_land", "Pct. Large Land-Owner", "Owns  $\geq$  3.01 hectares",
  "pct_head_illiterate", "Pct. Head Educ., Illiterate", "Percentage of household heads
  ↪ with educ. level: illiterate",
  "pct_head_lit_to_primary", "Pct. Head Educ., Lit.-Primary", "Percentage of heads with
  ↪ educ. level: literate-primary",
  "pct_head_secondary_plus", "Pct. Head Educ., Secondary+", "Percentage of heads with
  ↪ educ. level: above secondary",
```

```

"pct_pucca", "Pct. Pucca", "Percentage in pucca (permanent) homes",

# 2017-18
"npeople_1718", "Population", "Estimated via NSS sample weights",
"consumption_1718", "Consumption", "Average household monthly consumption expenditures
  ↳ (Rs.)",
"gini_cons_1718", "Gini of Consumption", "Gini coefficient of consumption",

# Changes
"consumption_pct_change", "$\\%\\Delta\\text{Consumption}$", "Percent change in
  ↳ consumption",
"gini_change", "$\\Delta\\text{Gini}^{\\text{Consumption}}$", "Change in the Gini
  ↳ coefficient of consumption"
)

# Summarize joined_df
# Process data
joined_stats <- joined_df %>%
  st_drop_geometry() %>%
  select(all_of(joined_meta$var)) %>%
  summarize(across(
    everything(),
    list(
      Min = ~min(.x, na.rm = TRUE),
      `1Q` = ~quantile(.x, .25, na.rm = TRUE),
      Med = ~median(.x, na.rm = TRUE),
      `3Q` = ~quantile(.x, .75, na.rm = TRUE),
      Max = ~max(.x, na.rm = TRUE),
      Mean = ~mean(.x, na.rm = TRUE),
      SD = ~sd(.x, na.rm = TRUE)
    ),
  ),

```

```

.names = "{.col}_{.fn}"
)) %>%
pivot_longer(
  cols = everything(),
  names_to = c("var", "stat"),
  # Only split on the final underscore, pulling off one of 7 stat-names
  names_pattern = "^(.*)_ (Min|1Q|Med|3Q|Max|Mean|SD)$"
) %>%
pivot_wider(names_from = stat, values_from = value) %>%
left_join(joined_meta, by = "var") %>%
arrange(match(var, joined_meta$var))

# Add number of observation column
joined_N <- joined_df %>%
  st_drop_geometry() %>%
  summarize(across(all_of(joined_meta$var),
    ~ sum(!is.na(.x)))) %>%
  pivot_longer(everything(),
    names_to = "var",
    values_to = "N")
joined_stats <- joined_stats %>%
  left_join(joined_N, by="var")

# Round and format
joined_stats <- joined_stats %>%
  mutate(
    across(
      Min:SD,
      ~ ifelse(
        var %in% c("npeople_0708", "npeople_1718"),
        # Round to 0 decimals, then add commas

```

```

    comma(round(.x, 0)),
    # Otherwise, keep two decimals
    sprintf("%.2f", .x)
  )
)
)# %>% mutate(across(Min:SD, ~ signif(.x, 3))) # For 3 significant digits

```

Chunk 28: Multicollinearity check

```

X <- c("EMIE", controls_needlag, controls_nolag) %>%
  reformulate() %>%
  model.matrix(data = joined_df)

# # Number of independent columns:
# qr(X)$rank
# # Num. total columns:
# ncol(X)
# # Recall: If qr(X)$rank < ncol(X), exact collinearity.

# Heuristic: kappa>30 implies severe multicollinearity
X_no_intercept <- X[, colnames(X) != "(Intercept)", drop = FALSE]
keep <- apply(X_no_intercept, 2, sd, na.rm = TRUE) > 0
X_scaled <- scale(X_no_intercept[, keep, drop = FALSE])
kappa_val <- kappa(X_scaled, exact = TRUE)

# # Check for linear dependence
# X_df <- as.data.frame(X_scaled)
# alias(lm(
#   consumption_pct_change ~ .,
#   data = cbind(consumption_pct_change = joined_df$consumption_pct_change, X_df)
# ))

```

```

# # Heuristic: VIF > 10 implies problematic collinearity
# vif(lm(
#   reformulate(
#     c("EMIE", controls_needlag, controls_nolag),
#     response = "consumption_pct_change"
#   ),
#   data = joined_df
# ))
#
# vif(model_consumption_iv)
# vif(model_gini_iv)

```

Chunk 29: Geospatial: Spatial autocorrelation tests

```

# Build row-standardized spatial weights (listw) object
# Recall: Number+order of rows in 2SLS models = number+order used for spatial weights
listw_2020 <- nb2listw(nb_2020, style = "W", zero.policy = TRUE) # TRUE for, say, islands

#Extract residuals from IV models
resid_cons <- residuals(model_consumption_iv)
resid_gini <- residuals(model_gini_iv)
resid_fscons <- residuals(first_stage_consumption)

# Global Moran's I on residuals
m_cons_resid <- moran.test(resid_cons, listw_2020, zero.policy = TRUE)
m_gini_resid <- moran.test(resid_gini, listw_2020, zero.policy = TRUE)
m_fscons_resid <- moran.test(resid_fscons, listw_2020, zero.policy = TRUE)

# # p-val's via Monte Carlo (moran.test() assumes asymptotic normality)
# set.seed(999)

```



```

# num_m = 9999
# mc <- moran.mc(resid_cons, listw_2020, nsim = num_m)
# plot(mc)
# mc$p.value

# # Before more controls were added in:
# m_cons_resid$p.value
# # 2.779572e-23
# m_gini_resid$p.value
# # 2.033012e-40
# m_fscons_resid$p.value
# # 1.189148e-105

# Moran's I on explanatory variable and IV
m_EMIE <- moran.test(joined_df$EMIE, listw_2020, zero.policy = TRUE)
m_wavg_ling_degrees <- moran.test(joined_df$wavg_ling_degrees, listw_2020, zero.policy =
  ↪ TRUE)

# m_EMIE$p.value
# # 8.990354e-180
# m_wavg_ling_degrees$p.value
# # 1.721903e-254
# All very extreme

# Each of the following are named; put them in unname() to get the raw number
# m_EMIE$statistic = z-score
# m_EMIE$estimate[1] = Moran's I statistic
# m_EMIE$estimate[2] = expected value of Moran's I under the null (no spatial
  ↪ autocorrelation i.e., randomized locations)
# m_EMIE$estimate[3] = variance under the null

```

```

# Test Moran's I on the response variables
m_cons <- moran.test(joined_df$consumption_pct_change, listw_2020, zero.policy=TRUE)
m_gini <- moran.test(joined_df$gini_change, listw_2020, zero.policy=TRUE)

# m_cons$p.value
# 1.608813e-26
# m_gini$p.value
# 8.51626e-22

# Repeat for controls which may have a strong degree of spatial autocorrelation
→ (infrastructure, poverty, etc.)

# View all the above statistics' p-values
# ls(pattern = "~m_") %>% sapply(.,function(name){get(name)$p.value}, simplify = TRUE)
→ %>% print

# All of these Moran's I stats are ridiculously, suspiciously high

```

Chunk 30: Spatial-2SLS model attempts

```

# Build spatial lags of dependent variables
joined_df$W_consY <- lag.listw(listw_2020, joined_df$consumption_pct_change,
→ zero.policy=TRUE)
joined_df$W_giniY <- lag.listw(listw_2020, joined_df$gini_change, zero.policy=TRUE)

# (Re)build the spatial lags of endogenous EMIE + IV
joined_df$W_EMIE <- lag.listw(listw_2020, joined_df$EMIE, zero.policy=TRUE)
joined_df$W_wLing <- lag.listw(listw_2020, joined_df$wavg_ling_degrees, zero.policy=TRUE)

```

```

# 2nd order lag
joined_df$W2_wLing <- lag.listw(listw_2020, joined_df$W_wLing, zero.policy=TRUE)

# (Re)build spatial lags of exogenous controls
controls <- c("npeople_0708", "nhouses_0708", "consumption_0708", "gini_cons_0708")
for(v in controls) {
  joined_df[[paste0("W_", v)]] <- lag.listw(listw_2020, joined_df[[v]], zero.policy=TRUE)
}

# Spatial-2SLS for consumption change, also instrumenting W_consY
model_sdm2sls_cons <- ivreg(
  make_iv_formula(
    dep = "consumption_pct_change",
    endog = c("W_consY", "EMIE", "W_EMIE"),
    exog = c(controls_needlag, controls_nolag, controls_lagged),
    inst = c(controls_needlag, controls_nolag, controls_lagged,
             "wavg_ling_degrees", "W_wLing", "W2_wLing")
  ),
  data = joined_df
)

model_sdm2sls_gini <- ivreg(
  make_iv_formula(
    dep = "gini_change",
    endog = c("W_giniY", "EMIE", "W_EMIE"),
    exog = c(controls_needlag, controls_nolag, controls_lagged),
    inst = c("wavg_ling_degrees", "W_wLing", "W2_wLing",
             controls_needlag, controls_nolag, controls_lagged)
  ),
  data = joined_df
)

```

```

# summary(model_sdm2sls_cons, diagnostics = TRUE)
# summary(model_sdm2sls_gini, diagnostics = TRUE)
# Don't work even when diagnostics = FALSE

# Region clustered standard errors

vcov_cluster_cons <- vcovCL(
  model_sdm2sls_cons,
  cluster = ~ region,
  data = joined_df
)
#HCO by default

vcov_cluster_gini <- vcovCL(
  model_sdm2sls_gini,
  cluster = ~ region,
  data = joined_df
)

# coeftest(model_sdm2sls_cons, vcov. = vcov_cluster_cons)
# coeftest(model_sdm2sls_gini, vcov. = vcov_cluster_gini)

```

Chunks 31–40: [Output for paper]

fig-ilo-trends (Figure 1)

```
# fig.pos='p': Placed on page with only floats (figs, tables, etc.)
files <- c(
  "Average Monthly Real Earnings Over Time - Total.png",
  "LFPR WPR and Unemployment for All Over Time.png",
  "Unemployment Rate By General Education.png"
)
paths <- file.path("580 Paper Images", files)

width_px <- 1300 # Reduce further if needed
imgs <- lapply(paths, function(p) image_read(p) |> image_scale(paste0(width_px)))

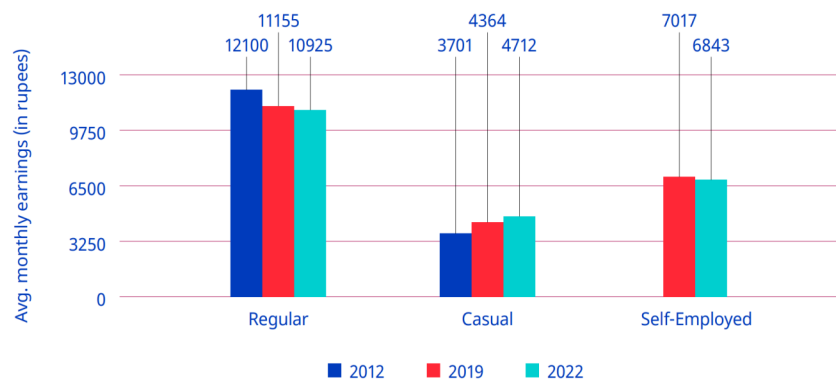
collage <- image_append(image_join(imgs), stack = TRUE)

out_path <- "ILO-fig.png"
image_write(collage, path = out_path)
knitr::include_graphics(out_path)
```

tab-probit-summary-numeric (Table 1)

```
kbl(
  sel_quant_stats %>%
    select(Variable=label, Min, `1Q`, Med, `3Q`, Max, Mean, SD, N),
  format = "latex",
  booktabs=TRUE,
  longtable=TRUE,
  label = "tab-probit-summary-numeric",
  caption="Summary Statistics for Enrollment Participation Model (Numeric Variables)",
  escape=FALSE
) %>%
```

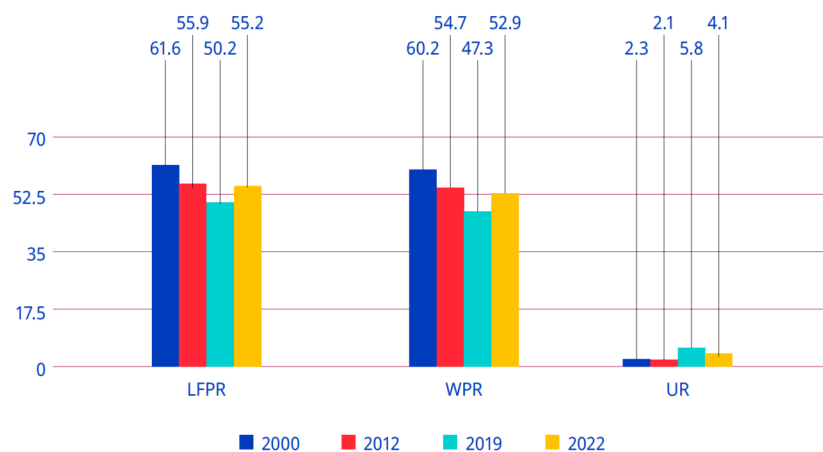
► Figure 2.8. Average monthly earnings of regular, casual and self-employed workers, 2012, 2019 and 2022 (rupees, at 2012 prices)



Note: Casual wages also included wages in public works, such as the Mahatma Gandhi National Rural Employment Guarantee. Wages were adjusted using the consumer price index-R and consumer price index-U by taking 2012 as the base year.

Source: Computed from various years of the Employment and Unemployment Survey data and the Periodic Labour Force Survey unit-level data.

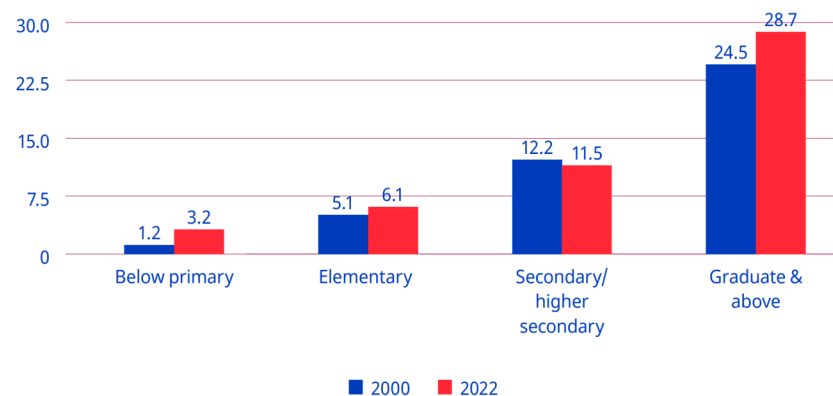
► Figure 2.1. Labour force participation rate, worker population ratio and unemployment rate (UPSS) among persons aged 15+ (rural and urban combined), 2000, 2012, 2019 and 2022 (%)



Note: LFPR=labour force participation rate; WPR=worker population ratio; UR=unemployment rate.

Source: Computed from various years of the Employment and Unemployment Survey data and the Periodic Labour Force Survey unit-level data.

► Figure 5.14. Unemployment rate for youths, by level of general education (UPSS), 2000 and 2022 (%)



Source: Computed from Employment and Unemployment Survey data for 2000 and Periodic Labour Force Survey data for 2022.

Figure 1: Trends in earnings, labor-force participation, and unemployment (ILO, 2024).

```

kable_styling(
  latex_options = c("hold_position", "repeat_header", "striped"),
  position = "center"
) %>%
# # Group district-level aggregate rows into single block
pack_rows("District-level aggregates:",
  which(sel_quant_stats$var=="dmean_num_IS_EDU_FREE"),
  nrow(sel_quant_stats)) %>%
footnote(
  general = paste(
    "`Min.' = minimum;",
    "`1Q' = first quartile;",
    "`Med.' = median;",
    "`3Q' = third quartile;",
    "`Max.' = maximum;",
    "`Mean' = arithmetic mean;",
    "`SD' = standard deviation;",
    "`N' = number of observations",
    collapse = " "
  ),
  escape = FALSE,
  threeparttable = TRUE,
  footnote_as_chunk = TRUE
) %>%
landscape()

```

Table 1: Summary Statistics for Enrollment Participation Model
(Numeric Variables)

Variable	Min	1Q	Med	3Q	Max	Mean	SD	N
Age	5.00	8.00	12.00	16.00	19.00	11.89	4.22	127246
Household size	1.00	4.00	5.00	7.00	30.00	5.91	2.40	127246
Enrollment cost (Rs.)	0.00	2,318.75	4,900.00	8,750.00	338,300.00	7,049.28	9,283.86	7184
District-level aggregates:								
Educ. free available? (Yes = 1)	0.00	0.48	0.69	0.85	1.00	0.65	0.23	127246
Tuition waived?	0.00	0.00	0.01	0.02	0.48	0.02	0.04	127246
Scholarship/Stipend?	0.00	0.01	0.05	0.20	0.90	0.13	0.17	127246
Textbooks received?	0.00	0.37	0.52	0.66	0.97	0.51	0.20	127246
Stationery received?	0.00	0.01	0.04	0.11	0.95	0.08	0.12	127246
Mid-day meal or more received?	0.02	0.33	0.42	0.55	0.90	0.43	0.17	127246

Note: ‘Min.’ = minimum; ‘1Q’ = first quartile; ‘Med.’ = median; ‘3Q’ = third quartile; ‘Max.’ = maximum; ‘Mean’ = arithmetic mean; ‘SD’ = standard deviation; ‘N’ = number of observations

tab-probit-summary-categorical (Table 2)

```
kbl(
  sel_cat_stats %>%
    select(Variable=label, Values, Mode, `"% Mode"`, `Least Freq.`, `"% Least Freq."`, N),
  format = "latex",
  col.names = c("Variable", "Values", "Mode", "Pct. Mode", "Least Freq.", "Pct. Least
    ↪ Freq.", "N"),
  booktabs=TRUE,
  longtable=TRUE,
  label = "tab-probit-summary-categorical",
  caption="Summary Statistics for Enrollment Participation Model (Categorical
    ↪ Variables)",
  escape=FALSE
) %>%
kable_styling(
  latex_options = c("hold_position", "repeat_header", "striped"),
  position = "center"
) %>%
footnote(
  general = paste(
    "`Values' = all possible values;",
    "`Mode' = most frequent value;",
    "`Pct. Mode' = Pct. of observations which take on the modal value;",
    "`Least Freq.' = least frequent value;",
    "`Pct. Least Freq.' = Pct. of observations which take on the least freq. value;",
    "`N' = number of observations.",
    collapse = " "
  ),
  escape = FALSE,
  threeparttable = TRUE,
  footnote_as_chunk = TRUE
) %>%
```

```
landscape() %>%  
# Set Variables column narrow, Values a bit wider, Mode smaller, etc.  
column_spec(1, width = "4cm") %>% # Variable  
column_spec(2, width = "6cm") %>% # Values  
column_spec(4, width = "2cm") %>% # Pct. Mode  
column_spec(6, width = "2cm") # Pct. Least Freq.
```

Table 2: Summary Statistics for Enrollment Participation Model
(Categorical Variables)

Variable	Values	Mode	Pct. Mode	Least Freq.	Pct. Least Freq.	N
Sex	Male, Female	Female	53.4	Male	46.6	127246
Religion	Hindu, Muslim, Christian, Sikh, Jain, Buddhist, Zoroastrian, Other	Hindu	75.2	Other	0.0	127246
Social group	Other, Scheduled Tribe, Scheduled Caste, Other Backward Class	Other Backward Class	39.5	Scheduled Tribe	14.2	127246
Urban	Rural, Urban	Rural	67.5	Urban	32.5	127246
Distance of nearest primary class	d<1km, 1km <= d <2kms, 2kms<= d <3kms, 3kms <= d <5kms, d>=5kms	d<1km	91.7	3kms <= d <5kms	0.1	126934
Father's education	Illiterate, Literate, no school, Literate, school < primary, Primary, Upper primary, Secondary, Higher secondary, Postsecondary+	Illiterate	55.3	Literate, no school	1.4	127179

Note: 'Values' = all possible values; 'Mode' = most frequent value; 'Pct. Mode' = Pct. of observations which take on the modal value; 'Least Freq.' = least frequent value; 'Pct. Least Freq.' = Pct. of observations which take on the least freq. value; 'N' = number of observations.

tab-probit-ame (Table 3)

```
# Print with kableExtra. Centered w/ booktabs style
kbl(
  reg_tbl_selection,
  format = "latex",
  booktabs = TRUE,
  longtable = TRUE,
  linesep = "",
  label = "tab-probit-ame",
  caption = "Average Marginal Effects for Enrollment Probit",
  align = c("l", "r", "r")
) %>%
kable_styling(
  latex_options = c("hold_position", "repeat_header", "striped"), # hold_position =
  ↪ Keep near chunk's position
  position = "center"
) %>%
add_header_above(c(" " = 1, "Enrolled in School (1 = yes)" = 2)) %>% # Dep. var. label
  ↪ above numeric vars
footnote(
  general = "Data from the 64th round of the NSS, ``Participation and Expenditure in
  ↪ Education'' in 2007-08. All standard errors are design-based, with PSUs clustered
  ↪ and nested within strata.",
  threeparttable = TRUE,
  escape = FALSE
)
```

Table 3: Average Marginal Effects for Enrollment Probit

Term	Enrolled in School (1 = yes)	
	Estimate	Std. Error
Age (years)	-0.029***	(0.000)
Female (ref: Male)	0.044***	(0.003)
Household size	-0.003***	(0.001)
Religion: Muslim (ref: Hindu)	-0.124***	(0.007)
Religion: Christian	-0.039**	(0.014)
Religion: Sikh	0.034	(0.026)
Religion: Jain	0.005	(0.012)
Religion: Buddhist	0.050*	(0.024)
Religion: Zoroastrian	0.105***	(0.032)
Religion: Other	0.254***	(0.002)
Social group: Scheduled Tribe (ref: Other)	-0.103***	(0.008)
Social group: Scheduled Caste	-0.076***	(0.006)
Social group: Other Backward Class	-0.030***	(0.005)
Urban (ref: Rural)	-0.006	(0.005)
Distance 1–2km (ref: <1km)	-0.068**	(0.025)
Distance 2–3km	-0.001	(0.010)
Distance 3–5km	0.007	(0.068)
Distance > 5km	0.043	(0.035)
Father’s educ.: Literate, no school (ref: Illiterate)	0.089***	(0.017)
Father’s educ.: Literate, school < primary	0.125***	(0.006)
Father’s educ.: Primary	0.141***	(0.005)
Father’s educ.: Upper primary	0.194***	(0.006)
Father’s educ.: Secondary	0.213***	(0.007)
Father’s educ.: Higher secondary	0.248***	(0.008)
Father’s educ.: Postsecondary+	0.239***	(0.011)
Educ. free available (ref: No)	-0.086***	(0.015)
Tuition waiver received	-0.060	(0.057)
Scholarship/Stipend received	0.022	(0.012)

Table 3: Average Marginal Effects for Enrollment Probit (*continued*)

Term	Estimate	Std. Error
Textbook(s) received	0.074***	(0.017)
Stationery received	0.015	(0.020)
Mid-day meal, etc. received	-0.017	(0.019)
Enrollment cost (Rs.)	0.000	(0.000)

Note:

Data from the 64th round of the NSS, “Participation and Expenditure in Education” in 2007–08. All standard errors are design-based, with PSUs clustered and nested within strata.

fig-district-maps-main (Figure 2)

```
knitr::include_graphics("collage1_map.png")
```

tab-iv-summary (Table 4)

```
kbl(
  joined_stats %>%
    select(Variable = label, Description = desc, Min, `1Q`, Med, `3Q`, Max, Mean, SD,
           N),
  booktabs=TRUE,
  longtable=TRUE, # longtable=T so table flows across pages
  label = "tab-iv-summary",
  caption="Summary Statistics for 2SLS Model",
  escape=FALSE, # escape=F so LaTeX labels can be used
  format = "latex"
) %>%
kable_styling(
  latex_options=c(
```

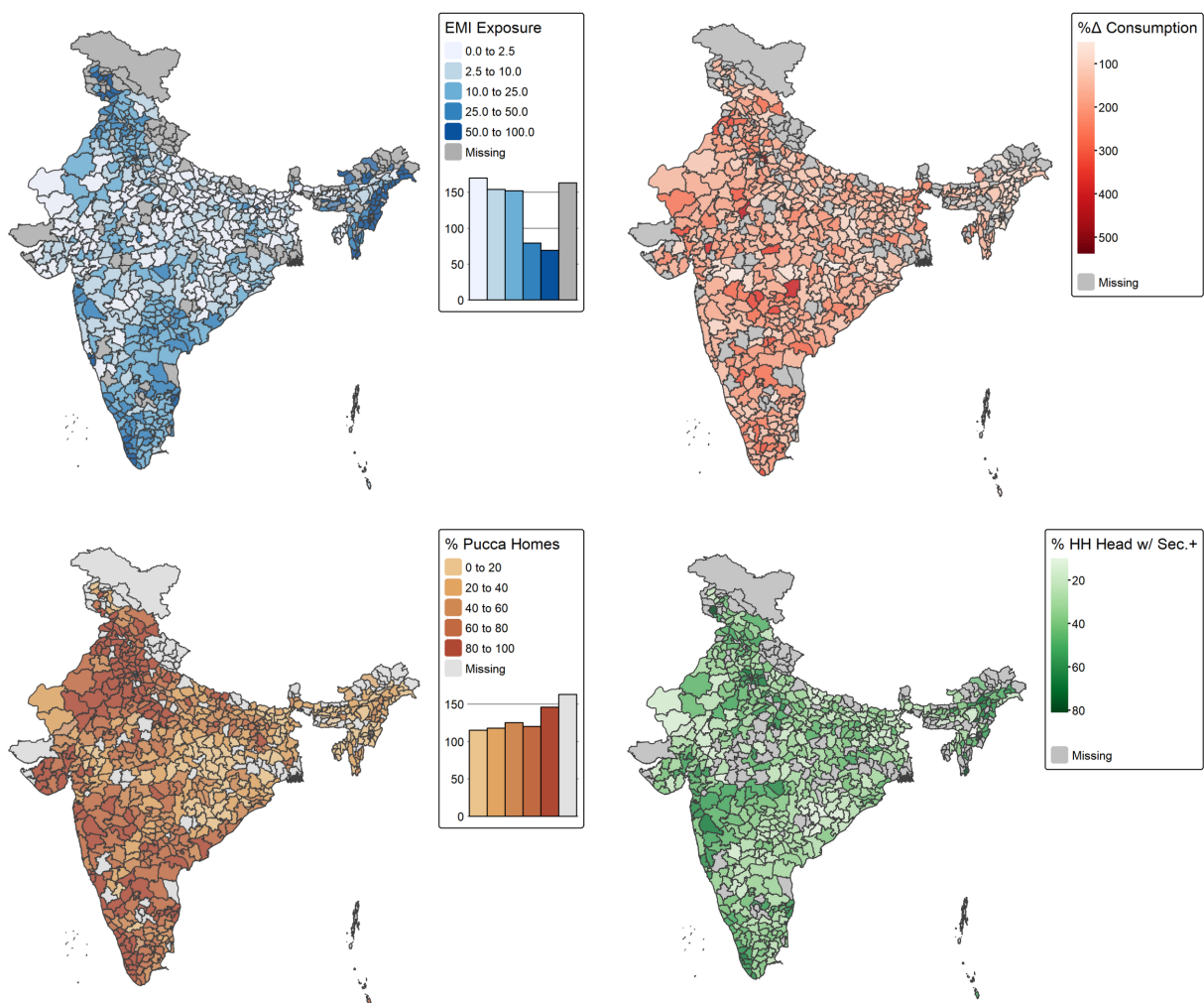


Figure 2: (Clockwise from top left) EMI exposure, consumption growth, pucca (permanent) housing, and household heads with secondary education or more. Data from the 64th round of the NSS 2007-08, “Participation and Expenditure in Education” and “Household Consumer Expenditure.”

```

    "repeat_header",
    "scale_down",
    "striped")
, full_width=FALSE
) %>% # Header repeats on each page, table scaled down so its full width fits on each
↪ page

## pack_rows, with row indices from joined_stats
pack_rows("From 2001:", 1, 1) %>%
pack_rows("From 2007-08:", 2, 22) %>%
pack_rows("From 2017-18:", 23, 25) %>%
pack_rows("From 2007-08 to 2017-18:", 26, 27) %>%
footnote(
  general = paste(
    "`Min.` = minimum;",
    "`1Q` = first quartile;",
    "`Med.` = median;",
    "`3Q` = third quartile;",
    "`Max.` = maximum;",
    "`Mean` = arithmetic mean;",
    "`SD` = standard deviation;",
    "`N` = number of observations",
    collapse = " "
  ),
  escape = FALSE,
  threeparttable = TRUE,
  footnote_as_chunk = TRUE
) %>%
landscape() %>%
# Narrow columns
column_spec(1, width = "4cm") %>% # Variable
column_spec(2, width = "6cm") # Description

```


Table 4: Summary Statistics for 2SLS Model

Variable	Description	Min	1Q	Med	3Q	Max	Mean	SD	N
From 2001:									
EMIE	EMI exposure	0.00	2.41	9.06	20.61	100.00	16.28	20.13	454
From 2007-08:									
Ling. Distance	Average linguistic distance of mother tongue from Hindi	0.00	0.06	1.77	4.17	5.00	2.14	1.93	454
Population	Estimated via NSS sample weights	31,492	1,056,344	1,662,361	2,654,010	9,922,640	1,979,019	1,389,793	454
Consumption	Average household monthly consumption expenditures (Rs.)	330.09	630.69	779.26	997.47	2923.14	861.53	339.82	454
Gini of Consumption	Gini coefficient of consumption	0.06	0.23	0.27	0.31	0.57	0.27	0.07	454
Pct. Urban	Percentage of people in an urban area	0.00	8.85	15.71	29.51	100.00	21.74	18.40	454
Avg. HH Size	Average household size	3.85	4.99	5.56	6.17	8.97	5.61	0.89	454
Dependency Ratio × 100	Ratio of dependents (0-14, 65+) to labor force (15-64), × 100	23.69	47.46	54.65	67.17	107.26	58.24	14.91	454
Pct. Female Head	Percentage of households with a female head	12.21	17.56	19.32	21.14	28.98	19.49	2.75	454
Pct. Hindu	Percentage of Hindus	0.00	77.61	89.17	95.24	100.00	80.66	22.73	454
Pct. Muslim	Percentage of Muslims	0.00	2.09	6.77	13.52	100.00	11.57	16.46	454
Pct. Other	Percentage not Hindu/Muslim	0.00	0.00	1.02	4.80	100.00	7.77	18.76	454
Pct. ST	Scheduled Tribe	0.00	0.00	2.02	11.05	100.00	11.79	21.43	454

Table 4: Summary Statistics for 2SLS Model (*continued*)

Variable	Description	Min	1Q	Med	3Q	Max	Mean	SD	N
Pct. SC	Scheduled Caste	0.00	11.53	18.37	25.82	48.86	18.75	10.23	454
Pct. OBC	Other Backward Class	0.00	22.18	44.93	60.73	88.32	42.10	22.76	454
Pct. Small Land-Owner	Owns 0.005–0.40 hectares	4.89	32.61	45.37	59.98	95.61	46.10	18.30	454
Pct. Med. Land-Owner	Owns 0.41–3.00 hectares	0.00	19.15	30.49	42.93	94.64	32.22	17.26	454
Pct. Large Land-Owner	Owns ≥ 3.01 hectares	0.00	0.00	1.78	6.18	43.73	4.25	6.16	454
Pct. Head Educ., Illiterate	Percentage of household heads with educ. level: illiterate	0.00	24.43	35.18	46.92	78.68	35.89	15.75	454
Pct. Head Educ., Lit.-Primary	Percentage of heads with educ. level: literate-primary	3.28	19.25	25.91	33.05	74.48	26.81	9.87	454
Pct. Head Educ., Secondary+	Percentage of heads with educ. level: above secondary	0.58	27.28	35.23	45.23	78.59	37.30	13.90	454
Pct. Pucca	Percentage in pucca (permanent) homes	0.00	32.68	58.79	81.64	100.00	56.85	27.89	454
From 2017-18:									
Population	Estimated via NSS sample weights	23,004	980,072	1,589,294	2,531,778	12,274,837	1,994,872	1,592,507	454
Consumption	Average household monthly consumption expenditures (Rs.)	771.50	1563.30	2077.24	2630.94	11031.46	2281.07	1030.43	454
Gini of Consumption	Gini coefficient of consumption	0.12	0.20	0.24	0.29	0.59	0.25	0.07	454

Table 4: Summary Statistics for 2SLS Model (*continued*)

Variable	Description	Min	1Q	Med	3Q	Max	Mean	SD	N
From 2007-08 to 2017-18:									
% Δ Consumption	Percent change in consumption	12.25	128.07	162.61	194.87	542.70	167.79	62.93	454
Δ Gini ^{Consumption}	Change in the Gini coefficient of consumption	-0.31	-0.07	-0.02	0.02	0.26	-0.02	0.07	454

Note: ‘Min.’ = minimum; ‘1Q’ = first quartile; ‘Med.’ = median; ‘3Q’ = third quartile; ‘Max.’ = maximum; ‘Mean’ = arithmetic mean; ‘SD’ = standard deviation; ‘N’ = number of observations

fig-regions-linguistic-distance (Figure 3)

```
knitr::include_graphics("collage2_map.png")
```

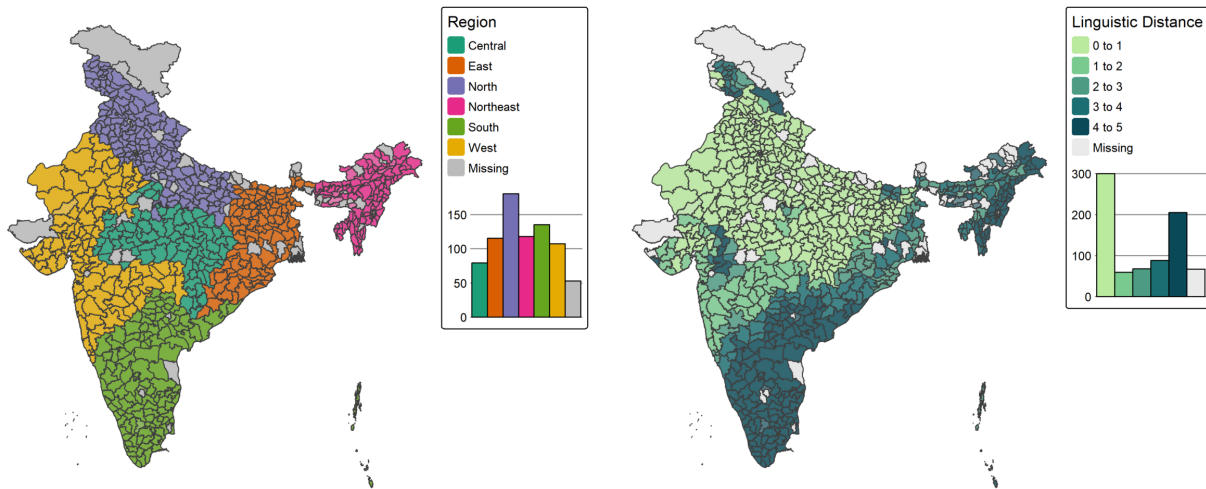


Figure 3: From left to right: regions of India and linguistic distance from Hindi. Data from the 2001 Census of India.

tab-first-stage-consumption (Table 5)

```
# Mapping for covariate labels
var_labels <- c(
  "EMIE" = "EMI exposure (fitted)",
  "wavg_ling_degrees" = "Linguistic distance",
  "consumption_0708" = "Consumption (2007-08)",
  "gini_cons_0708" = "Gini of Consumption (2007-08)",
  "avg_IMR" = "Average IMR",
  "pct_urban" = "Pct. Urban (ref: Rural)",
  "avg_hh_size" = "Average HH size",
  "dependency_ratio" = "Dependency ratio × 100",
  "pct_fem_head" = "Pct. Female head",
  "pct_hindu" = "Pct. Hindu (ref: Other)",
  "pct_muslim" = "Pct. Muslim",
  "pct_st" = "Pct. Scheduled Tribe (ref: Other)",
```

```

    "pct_sc" = "Pct. Scheduled Caste",
    "pct_obc" = "Pct. OBC",
    "pct_small_land" = "Pct. Small Land-owner (ref: No Land)",
    "pct_medium_land" = "Pct. Medium Land-owner",
    "pct_large_land" = "Pct. Large Land-owner",
    "pct_head_lit_to_primary" = "Pct. Head Educ., Lit.-Primary (ref: Illiterate)",
    "pct_head_secondary_plus" = "Pct. Head Educ., Secondary+",
    "(Intercept)" = "Intercept"
)

#### Test instrument's strength ####

# Run cluster-robust Wald F-test
vcov_partialf <- vcovCL(first_stage_consumption, cluster = joined_df$state_20)

partial_F_test <- linearHypothesis(
  first_stage_consumption,
  "wavg_ling_degrees = 0",
  vcov. = vcov_partialf,
  test = "F"
) # Wald F-stat with robust vcov + one endogenous regressor "is equivalent to the
  ↪ Kleibergen-Paap statistic" (https://lrberge.github.io/fixest/reference/fitstat.html)

# Extract F-stat and p-val
partial_f <- partial_F_test[["F"]][2]
partial_p <- partial_F_test[["Pr(>F)"]][2]

# Create asterisks based on p-val
stars <- function(p) {
  if (p < 0.001) return("***")
  else if (p < 0.01) return("**")

```

```

else if (p < 0.05) return("*")
else return("")
}

# Add custom GOF row for the partial F-stat
custom_gof_row <- data.frame(
  term = "Instrument's F-Statistic",
  estimate = paste0(formatC(partial_f, digits = 2, format = "f"), stars(partial_p)),
  stringsAsFactors = FALSE
)

# Table with styling and stats
modelsummary(
  first_stage_consumption,
  coef_map = var_labels,
  vcov = vcov_first_stage_consumption,
  gof_map = list(
    list(row = "nobs", clean = "Observations", fmt = 0),
    list(row = "r.squared", clean = "$R^2$", fmt = 3),
    list(row = "adj.r.squared", clean = "Adjusted $R^2$", fmt = 3),
    list(row = "sigma", clean = "Residual Std. Error", fmt = 3),
    list(row = "statistic", clean = "Model's $F$-Statistic", fmt = 2)
  ),
  stars = c('*' = .05, '**' = .01, '***' = .001),
  fmt = 3,
  title = "First-Stage Regression: EMI Exposure on Linguistic Distance",
  output = "kableExtra",
  escape = FALSE,
  notes = list(
    "Standard errors clustered by state in parentheses."
  ),

```

```

add_rows = custom_gof_row
) %>%
kable_styling(latex_options = c(
  "hold_position",
  "repeat_header",
  "striped",
  "longtable"),
position = "center",
full_width = FALSE
) %>%
add_header_above(c(" " = 1, "EMI Exposure" = 1))

```

tab-second-stage-consumption-iv (Table 6)

```

modelsummary(
  model_consumption_iv,
  coef_map = var_labels,
  vcov = vcov_model_consumption_iv,
  gof_map = list(
    list(row = "nobs", clean = "Observations", fmt = 0),
    list(row = "r.squared", clean = "$R^2$", fmt = 3),
    list(row = "adj.r.squared", clean = "Adjusted $R^2$", fmt = 3),
    list(row = "sigma", clean = "Residual Std. Error", fmt = 3),
    list(row = "waldtest", clean = "F-Statistic", fmt = 2)
  ),
  stars = c('*' = .05, '**' = .01, '***' = .001),
  fmt = 3,
  title = "Second-Stage Regression: Consumption Growth on EMIE (Fitted)",
  output = "kableExtra",
  escape = FALSE,
  notes = list(
    "Standard errors clustered by state in parentheses."
  )
)

```

Table 5: First-Stage Regression: EMI Exposure on Linguistic Distance

	EMI Exposure
	(1)
Linguistic distance	2.945** (0.949)
Consumption (2007-08)	0.018*** (0.005)
Gini of Consumption (2007-08)	-40.452* (19.986)
Pct. Urban (ref: Rural)	0.245** (0.086)
Average HH size	-0.576 (2.059)
Dependency ratio $\times$ 100	-0.144 (0.089)
Pct. Female head	-0.415 (0.555)
Pct. Hindu (ref: Other)	-0.273* (0.109)
Pct. Muslim	-0.085 (0.183)
Pct. Scheduled Tribe (ref: Other)	0.015 (0.098)
Pct. Scheduled Caste	-0.012 (0.114)
Pct. OBC	-0.033 (0.088)
Pct. Small Land-owner (ref: No Land)	0.156* (0.078)
Pct. Medium Land-owner	0.167 (0.118)
Pct. Large Land-owner	0.115 (0.135)
Pct. Head Educ., Lit.-Primary (ref: Illiterate)	-0.319** (0.123)
Pct. Head Educ., Secondary+	0.193 (0.121)
Intercept	33.076 (27.314)
Observations	454
R^2	0.630
Adjusted R^2	0.615
Instrument's F-Statistic	9.63**

* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Standard errors clustered by state in parentheses.

),

`add_rows = NULL`


```

) %>%
  kable_styling(
    latex_options = c("hold_position", "repeat_header", "striped", "longtable"),
    position = "center",
    full_width = FALSE
  ) %>%
  add_header_above(c(" " = 1, "Consumption Growth" = 1))

```

fig-district-carveouts-shifts (Figure 4)

```
plt_district_carveouts_shifts
```

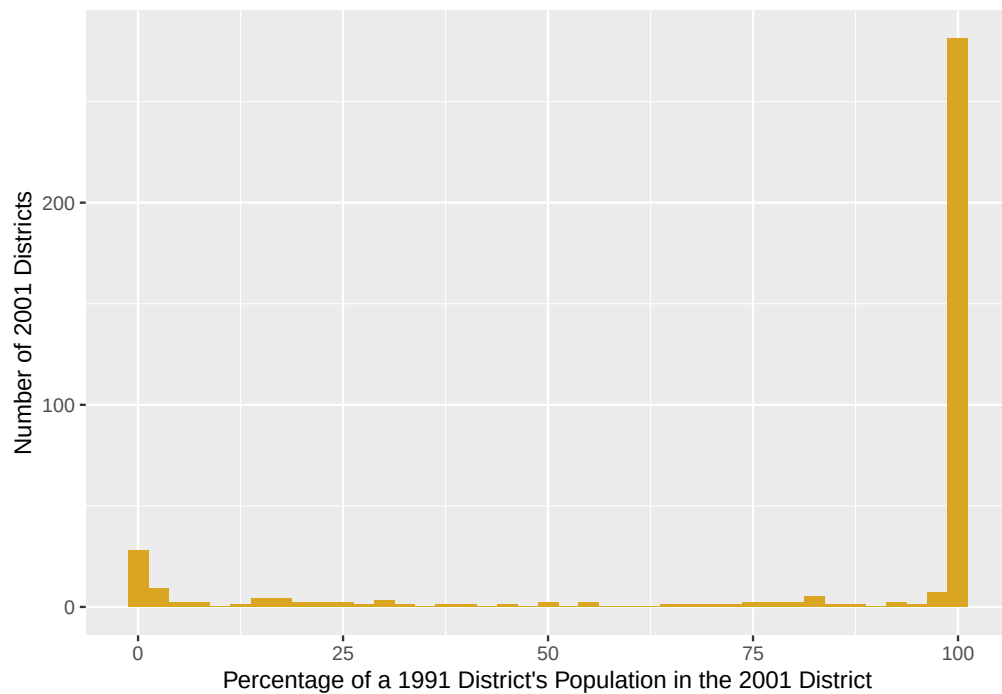


Figure 4: Number of 2001 districts which absorbed a percentage of a 1991 district's population via name change, clean merger, carve-out, or border shift. Data from Kumar & Somanathan (2016).

Table 6: Second-Stage Regression: Consumption Growth on EMIE (Fitted)

	Consumption Growth
	(1)
EMI exposure (fitted)	0.201 (0.710)
Consumption (2007-08)	-0.104*** (0.027)
Gini of Consumption (2007-08)	-11.654 (68.983)
Pct. Urban (ref: Rural)	1.240** (0.438)
Average HH size	-5.080 (6.090)
Dependency ratio $\times$ 100	-0.419 (0.263)
Pct. Female head	-1.807 (2.167)
Pct. Hindu (ref: Other)	-0.404 (0.232)
Pct. Muslim	-0.786** (0.277)
Pct. Scheduled Tribe (ref: Other)	-0.857*** (0.238)
Pct. Scheduled Caste	-0.223 (0.293)
Pct. OBC	-0.615*** (0.177)
Pct. Small Land-owner (ref: No Land)	-0.169 (0.313)
Pct. Medium Land-owner	-0.717 (0.459)
Pct. Large Land-owner	0.567 (0.395)
Pct. Head Educ., Lit.-Primary (ref: Illiterate)	-0.384 (0.466)
Pct. Head Educ., Secondary+	-0.494* (0.251)
Intercept	457.536*** (86.523)
Observations	454
R^2	0.232
Adjusted R^2	0.202

* $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$

Standard errors clustered by state in parentheses.